



INSTITUTO SUPERIOR POLITÉCNICO DE TECNOLOGIAS E CIÊNCIAS
DEPARTAMENTO DE ENGENHARIA E TECNOLOGIAS
CURSO DE ENGENHARIA INFORMÁTICA

COMPILADOR DE JAVA
MANUAL DO PROGRAMADOR
MANUAL DO UTILIZADOR

Nome: Abel Nkele Canas

Nº Processo: 20231051

Turma: M3

Ano: 3º ano

ÍNDICE

ÍNDICE.....	2
1. INTRODUÇÃO.....	5
2. DESENVOLVIMENTO	7
2.1. Escolha e especificação da linguagem em estudo	7
2.2. Construção do Analisador Léxico (Scanner).....	7
2.3. Construção do Analisador Sintático (Parser)	9
2.4. Construção da Tabela de Símbolos.....	12
2.5. Construção do Analisador Semântico.....	14
2.6. Integração das três fases e exemplo de execução completa.....	16
3. CONCLUSÃO	19
4. REFERÊNCIAS BIBLIOGRÁFICAS.....	20
5. APÊNDICES.....	21
Apêndice A — Manual do Programador.....	21
Manual do Programador.....	21
MINI-COMPILADOR para um subconjunto da linguagem Java	21
1. INTRODUÇÃO AO MANUAL	21
2. VISÃO GERAL DA ARQUITETURA	22
3. ORGANIZAÇÃO DO CÓDIGO-FONTE	23
4. O PACOTE LEXER — ANALISADOR LÉXICO	23
4.1. Tipo (enum).....	23
4.2. Token (classe)	24
4.3. LexError (exceção).....	24
4.4. Lexer (classe principal).....	24
5. O PACOTE PARSER — ANALISADOR SINTÁTICO E AST	25
5.1. Metodologia de construção da gramática	25
5.2. A Árvore Sintática Abstrata (AST).....	26
5.3. Preenchimento da Tabela de Símbolos	27
5.4. Alocação recursiva de sub-escopos.....	27
6. O PACOTE SYMBOL — TABELA DE SÍMBOLOS	27
6.1. SymbolEntry	27
6.2. SymbolTable.....	28
7. O PACOTE SEMANTIC — ANALISADOR SEMÂNTICO	29

7.1. Type — representação de tipos	29
7.2. Estrutura da travessia.....	30
7.3. Mapeamento direto às alíneas do enunciado	30
7.4. SemanticError	31
7.5. Limitações assumidas (documentadas no próprio código).....	31
8. O PACOTE MAIN — PONTO DE ENTRADA.....	32
9. FLUXO DE EXECUÇÃO COMPLETO (PASSO A PASSO)	32
10. COMO COMPILAR O PROJETO	33
Opção A — via NetBeans.....	33
Opção B — via linha de comandos (sem IDE).....	33
11. COMO EXECUTAR O PROJETO	33
12. COMO ESTENDER O COMPILADOR (GUIA PRÁTICO)	33
12.1. Adicionar uma nova verificação semântica	34
12.2. Adicionar um novo tipo de instrução (statement) à linguagem	34
12.3. Adicionar suporte a uma nova classe da biblioteca padrão	34
12.4. Depurar um caso em que a Tabela de Símbolos não tem a entrada esperada	34
13. TRATAMENTO E RECUPERAÇÃO DE ERROS	35
13.1. Erros léxicos.....	35
13.2. Erros sintáticos — modo pânico em três níveis	35
13.3. Erros semânticos	36
14. Convenções de código adotadas	36
15. TESTES E FICHEIROS DE EXEMPLO	37
16. LIMITAÇÕES CONHECIDAS E TRABALHO FUTURO	37
17. REFERÊNCIA RÁPIDA DE CLASSES (API)	37
18. CONCLUSÃO.....	39
19. Referências Bibliográficas	40
Apêndice B — Manual do Utilizador	40
MANUAL DO UTILIZADOR.....	41
MINI-COMPILADOR para um subconjunto da linguagem Java	41
1. O QUE É ESTE PROGRAMA	41
2. REQUISITOS	42
3. INSTALAÇÃO	42

Opção A — Abrir no NetBeans (mais simples)	42
Opção B — Compilar manualmente pela linha de comandos	42
4. COMO EXECUTAR	42
Sobre acentos na consola do Windows	42
5. FORMATO DO FICHEIRO DE ENTRADA	43
6. COMO LER A SAÍDA DO PROGRAMA.....	43
6.1. Nota para quem quiser depurar o Lexer (avançado)	44
6.2. A Tabela de Símbolos	44
6.3. Erros sintáticos vs. erros semânticos	45
7. SUBCONJUNTO DA LINGUAGEM SUPORTADO	45
O que o analisador semântico verifica, especificamente	45
8. EXEMPLOS COMPLETOS PASSO A PASSO	46
8.1. Exemplo sem erros	46
8.2. Exemplo com erros semânticos (todas as verificações).....	46
8.3. Exemplo com arrays e for-each	48
8.4. Exemplo com erro sintático.....	48
9. MENSAGENS DE ERRO MAIS COMUNS E O QUE SIGNIFICAM.....	49
10. PERGUNTAS FREQUENTES	49
11. LIMITAÇÕES E O QUE O PROGRAMA NÃO FAZ.....	50
12. RESOLUÇÃO DE PROBLEMAS (TROUBLESHOOTING)	50
13. GLOSSÁRIO DE TERMOS TÉCNICOS	51
14. REFERÊNCIA RÁPIDA: PALAVRAS-RESERVADAS E OPERADORES RECONHECIDOS	53
14.1. Palavras-reservadas.....	53
14.2. Literais especiais	53
14.3. Operadores e delimitadores	53
14.4. Categorias de literais	54
15. CONCLUSÃO.....	55
16. REFERÊNCIAS BIBLIOGRÁFICAS.....	56

1. INTRODUÇÃO

A construção de um compilador é um dos exercícios mais completos que um estudante de Engenharia Informática pode realizar, pois obriga a articular, num só sistema, conhecimentos de teoria de linguagens formais e autómatos, estruturas de dados, algoritmos de recuperação de erros e organização de software. Este documento descreve o trabalho realizado no âmbito do Exame Normal de Compiladores, cujo objetivo específico é a construção da terceira e última fase do front-end de um compilador — o **Analizador Semântico** — integrada com as duas fases já desenvolvidas anteriormente: o **Analizador Léxico** e o **Analizador Sintático**.

Um compilador tem como função transformar um programa escrito numa linguagem de alto nível (a linguagem-fonte) num programa equivalente, executável ou numa representação intermédia. Antes de qualquer geração de código, porém, o compilador precisa de garantir que o programa de entrada é **válido** em três níveis crescentes de exigência:

- ao nível **léxico**, garantindo que toda a sequência de caracteres do ficheiro-fonte pode ser agrupada em unidades reconhecíveis (tokens): identificadores, palavras reservadas, literais, operadores e delimitadores;
- ao nível **sintático**, garantindo que a sequência de tokens respeita a gramática da linguagem, produzindo uma **Árvore Sintática Abstrata (AST)** que representa a estrutura hierárquica do programa;
- ao nível **semântico**, garantindo que o programa, apesar de gramaticalmente correto, também faz sentido: que toda a variável usada foi previamente declarada, que não há duas declarações do mesmo nome no mesmo escopo, que os tipos envolvidos em operações e atribuições são compatíveis entre si, e que as estruturas de controlo de fluxo recebem os argumentos do tipo esperado.

O projeto aqui descrito — a que chamámos **MINI-COMPILADOR** — implementa estas três fases para um **subconjunto realista da linguagem Java**, escolhido pela sua familiaridade (é a linguagem de programação predominantemente usada ao longo do curso), pela riqueza da sua gramática (suficientemente complexa para justificar técnicas de eliminação de ambiguidade e de recursividade à esquerda, exigidas pelo enunciado) e pelo seu sistema de tipos estático, que se presta naturalmente à implementação das verificações semânticas pedidas.

Os objetivos concretos deste trabalho, decorrentes diretamente do enunciado do exame, foram:

1. Construir um **Analizador Léxico** (scanner) capaz de reconhecer todos os tokens do subconjunto de Java escolhido, com base em autómatos finitos determinísticos e expressões regulares informais para cada categoria de token.
2. Construir um **Analizador Sintático** (parser) a partir de uma gramática preparada especificamente para ser processada por descida recursiva, removendo ambiguidades (o clássico problema do dangling else) e recursividade à esquerda (nas produções de expressões).

3. Construir um **Analizador Semântico** que, a partir da Árvore Sintática e da Tabela de Símbolos preenchidas pelas fases anteriores, verifique: (a) uso de variáveis não declaradas; (b) declarações duplicadas no mesmo escopo; (c) incompatibilidade de tipos de dados; (d) tipos incorretos nos argumentos de chamadas de métodos; (e) atribuições de tipos incompatíveis a variáveis; e (f) argumentos inválidos nas estruturas de controlo (condicionais e ciclos).

O restante deste documento está organizado da seguinte forma: a secção 2 ("Desenvolvimento") descreve, com o necessário detalhe técnico, a metodologia usada em cada uma das três fases, mostrando como as decisões de implementação decorrem diretamente da teoria de compiladores lecionada na disciplina; a secção 3 apresenta as conclusões do trabalho; a secção 4 lista as referências bibliográficas consultadas; e os apêndices reproduzem, na íntegra, o Manual do Programador e o Manual do Utilizador do MINI-COMPILADOR.

2. DESENVOLVIMENTO

2.1. Escolha e especificação da linguagem em estudo

A linguagem escolhida para este projeto foi um **subconjunto da linguagem Java (SE 17)**. A escolha recaiu sobre Java por três razões principais:

- É a linguagem com que o autor tem maior familiaridade prática, reduzindo o risco de erros de especificação da gramática;
- É uma linguagem **fortemente tipada e orientada a objetos**, o que torna a fase de análise semântica (o foco central deste exame) particularmente rica: existe verificação de tipos em atribuições, chamadas de método, expressões aritméticas, estruturas de controlo, e existe um conceito claro de escopo (classe, método, bloco) que se presta a uma tabela de símbolos hierárquica;
- A gramática de Java contém, de forma natural, os dois problemas clássicos que o enunciado pede explicitamente para tratar: **ambiguidade** (o problema do dangling else, presente em qualquer linguagem tipo C/Java com if/else opcional) e **recursividade à esquerda** (presente em todas as produções de expressões binárias: aditivas, multiplicativas, relacionais, lógicas, etc.).

O subconjunto implementado cobre: declaração de pacotes e importações; classes e interfaces (incluindo aninhadas, com modificadores, herança de interface via implements e de classe via extends, e sintaxe de genéricos <T extends X>); campos, métodos e construtores; todos os tipos primitivos de Java, String, tipos definidos pelo utilizador e arrays de qualquer dimensão; todas as instruções de controlo de fluxo (if/else, while, do-while, for clássico e for-each, switch, try/catch/finally, break, continue, return, throw, assert, synchronized); e a totalidade dos operadores de expressão de Java, incluindo atribuições compostas, instanceof, o operador ternário, incremento/decremento e conversões explícitas de tipo (cast).

A especificação completa da gramática, em notação BNF, está documentada no ficheiro gramática.txt incluído no código-fonte entregue, e é usada como referência ao longo de toda a secção seguinte.

2.2. Construção do Analisador Léxico (Scanner)

2.2.1. Metodologia

O analisador léxico (classe Lexer, pacote com.compiler.lexer) foi construído como um **Autómato Finito Determinístico (AFD)** explícito. Em vez de recorrer a uma biblioteca geradora de scanners (como o Lex/Flex ou o ANTLR), optou-se por implementar manualmente:

1. Um conjunto de **estados** do autómato, representados por um enum Java (Estado, com estados Q0 a Q63), onde Q0 é sempre o estado inicial de reconhecimento de um novo token;
2. Uma **função de transição** transicao(Estado, char): Estado, que implementa, para cada estado e cada categoria de carácter de entrada (letra, dígito, símbolo

específico), qual o próximo estado do autômato — ou null quando não existe transição válida, indicando o fim do reconhecimento do lexema corrente;

3. Uma função `isEstadoFinal(Estado)`: boolean que identifica quais os estados de aceitação do autômato, isto é, em que ponto um lexema acumulado já constitui um token válido e completo.

Esta abordagem — construir o autômato "à mão", estado a estado — foi escolhida deliberadamente para tornar explícita, no próprio código, a correspondência entre a teoria de autômatos finitos lecionada na disciplina (diagrama de estados, função de transição, estados de aceitação) e a implementação. Cada categoria de token corresponde a um "ramo" percorrido dentro do mesmo autômato global:

- **Identificadores e palavras reservadas:** reconhecidos a partir do estado Q8 (sequência de letras, dígitos, `_` ou `$`). No fecho do lexema, uma verificação lexicográfica (não fonética) contra duas listas — `keywordsArray` (as 50 palavras reservadas de Java) e `literalsArray` (`true`, `false`, `null`) — decide a categoria final do token: `PALAVRA_RESERVADA`, `LITERAL_ESPECIAL` ou `IDENTIFICADOR`.
- **Números:** o estado Q1 acumula dígitos; a transição para Q3→Q4 reconhece a parte decimal (`.dígitos`); os sufixos `L/l` (estado Q2) e `F/f/D/d` (estado Q5) permitem distinguir, no fecho do token, entre `LITERAL_INTEIRO`, `LITERAL_LONG`, `LITERAL_FLOAT` e `LITERAL_DOUBLE`.
- **Strings e caracteres:** geridos por sub-rotinas dedicadas (`processarString`, `processarChar`) que tratam corretamente sequências de escape (`\`, `\\`, `\n`, etc.) sem as interpretar — apenas as preservam no lexema — e que lançam um erro léxico (`LexError`) caso o literal não seja fechado antes do fim da linha (para strings) ou do ficheiro.
- **Comentários:** comentários de linha (`//`) e de bloco (`/* ... */`, com suporte a aninhamento através de um contador `nestedLevel`) são reconhecidos e **descartados** sem gerar token, através de sub-rotinas que avançam a posição de leitura sem produzir saída para o parser.
- **Operadores:** os operadores de um só carácter (`+`, `-`, `<`, `=`, etc.) são tratados pela tabela de transições genérica (Q6 como estado intermédio); no entanto, para evitar uma explosão do número de estados necessários para cobrir todas as combinações de 2 a 4 caracteres de um operador Java (`>>>=` é o caso mais extremo), optou-se por resolver os operadores compostos através de uma função de **lookahead direto** (`processarOperadorEspecifico`), que examina os 1 a 3 caracteres seguintes ao primeiro símbolo do operador. Esta é uma simplificação de engenharia sobre o autômato "puro", plenamente equivalente em poder de reconhecimento, mas muito mais legível do que expandir o autômato para dezenas de estados adicionais só para operadores.

2.2.2. Expressões regulares informais por categoria de token

Ainda que o scanner não use uma biblioteca de expressões regulares em tempo de execução (a correspondência é feita diretamente pelo autômato), a gramática léxica de

cada categoria de token pode ser descrita pelas seguintes expressões regulares equivalentes, que serviram de base ao desenho dos estados do autómato:

```
IDENTIFICADOR ::= [A-Za-z_$][A-Za-z0-9_$]*
LITERAL_INTEIRO ::= [0-9]+
LITERAL_LONG ::= [0-9]+[iL]
LITERAL_DOUBLE ::= [0-9]+"."[0-9]+([dD])?
LITERAL_FLOAT ::= [0-9]+"."[0-9]+[fF]
LITERAL_TEXTO ::= \"(^[\\n] | \\. )*\
LITERAL_CARACTERE ::= '([\\n] | \\. )'
COMENTARIO_LINHA ::= \"//\" [^\\n]*
COMENTARIO_BLOCO ::= \"/*\" ( qualquer carácter, incl. aninhado )* \"*/\"
```

2.2.3. Tratamento de erros léxicos

Dois tipos de situação são tratados de forma distinta:

- **Caracteres não reconhecidos** (nenhuma transição válida a partir de Q0, ex.: um símbolo estranho como @ fora de contexto ou um emoji): produzem um token DESCONHECIDO, que é reportado como aviso em System.err e **descartado**, permitindo que o scanner continue a processar o resto do ficheiro sem interromper a análise.
- **Literais mal fechados** (string, caractere ou comentário de bloco que chegam ao fim da linha/ficheiro sem o delimitador de fecho): são considerados erros fatais e lançam a exceção LexError, pois não existe, dentro do próprio literal, um ponto seguro de sincronização.

2.3. Construção do Analisador Sintático (Parser)

2.3.1. Preparação da gramática: eliminação de ambiguidades e de recursividade à esquerda

A técnica escolhida para o analisador sintático foi a de um **parser descendente recursivo** (recursive descent parser), que exige que a gramática de entrada seja preparada de forma a poder ser reconhecida por antecipação limitada de tokens (lookahead), sem retrocesso (backtracking). Duas transformações foram necessárias sobre a gramática "natural" de Java:

a) Eliminação da ambiguidade do "dangling else". A gramática natural de uma instrução condicional em Java —

```
<instrucaoIf> ::= "if" "(" <expressao> ")" <instrucao> ("else" <instrucao>)?
```

— é ambígua: numa sequência if (a) if (b) S1 else S2, não é sintaticamente óbvio se o else pertence ao primeiro ou ao segundo if. A solução clássica (adotada também pelo próprio Java) é fazer o else ligar-se sempre ao if aberto mais próximo. Para tornar isto **não ambíguo na gramática**, foi introduzida uma segunda produção, <instrucaoNoElse>, que restringe as instruções permitidas no ramo "then" de um if com else, de forma a nunca permitir aí um if "em aberto":

```
<instrucaolf> ::= "if" "(" <expressao> ")" <instrucaoNoElse> ("else" <instrucao>)?  
<instrucaoNoElse> ::= ... | <instrucaolfNoElse> | ...  
<instrucaolfNoElse> ::= "if" "(" <expressao> ")" <instrucaoNoElse>
```

Com esta reescrita, a gramática deixa de ter ambiguidade: cada `else` corresponde exatamente a um `if`, determinado estruturalmente pela própria gramática, e não por uma convenção aplicada arbitrariamente no código do parser.

b) Eliminação da recursividade à esquerda nas expressões. As expressões binárias de Java são naturalmente definidas de forma recursiva à esquerda, por exemplo:

```
<expressaoAditiva> ::= <expressaoAditiva> ("+" | "-") <expressaoMultiplicativa>  
| <expressaoMultiplicativa>
```

Uma gramática recursiva à esquerda **não pode** ser reconhecida por um parser descendente recursivo direto, pois a chamada da função `parseExpressaoAditiva()` invocaria a si própria antes de consumir qualquer token, entrando em recursão infinita. A solução aplicada, padrão em construção de compiladores, foi reescrever cada produção recursiva à esquerda na forma equivalente **iterativa à direita**:

```
<expressaoAditiva> ::= <expressaoMultiplicativa> (("+" | "-")  
<expressaoMultiplicativa>)*
```

Esta transformação foi aplicada sistematicamente a **todos** os níveis de precedência de expressões da linguagem, resultando na seguinte hierarquia de métodos de parsing (do menor para o maior nível de precedência, cada um delegando no nível imediatamente abaixo e, de seguida, repetindo num laço `while` o consumo do operador do seu próprio nível):

Esta hierarquia de métodos implementa, de forma totalmente explícita, o mesmo resultado de um parser de precedência de operadores (Pratt parser), mas sem necessidade de uma tabela de precedências consultada em tempo de execução — cada nível de precedência é, literalmente, um método da hierarquia, o que torna o comportamento do parser mais fácil de verificar e depurar.

Cada método chama o nível seguinte e, num laço, consome o(s) operador(es) do seu próprio nível — eliminando a recursividade à esquerda

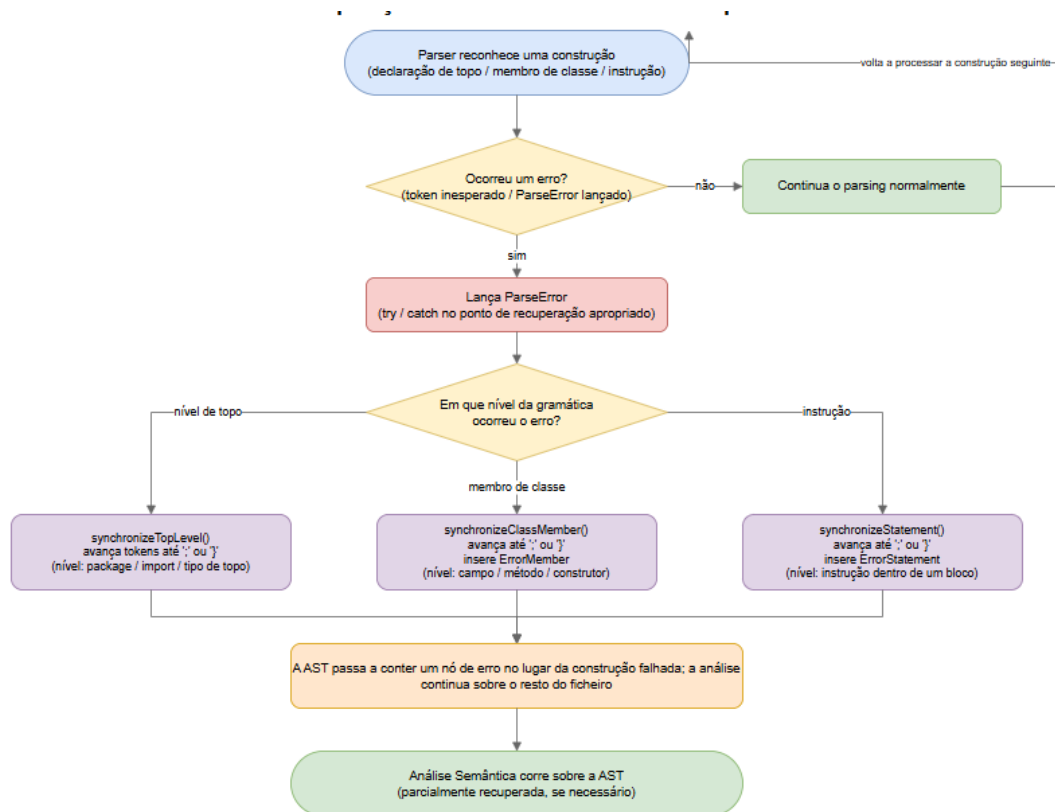


2.3.2. Construção da Árvore Sintática Abstrata (AST)

Durante o reconhecimento da gramática, o parser constrói simultaneamente uma **Árvore Sintática Abstrata**: cada regra da gramática corresponde a uma classe de nó Java (ClassDeclaration, MethodDeclaration, IfStatement, BinaryExpression, MethodInvocationExpression, entre mais de 40 classes de nó), todas derivando de uma classe base Node que regista a linha e a coluna onde a construção começa no ficheiro-fonte — informação depois reaproveitada pelo analisador semântico para localizar exatamente onde reportar cada erro.

Um cuidado especial foi colocado na **recuperação de erros sintáticos**: em vez de abortar a análise assim que surge um token inesperado, o parser implementa **recuperação em modo pânico** (panic-mode error recovery), com três pontos de

sincronização, cada um adequado ao nível estrutural em que o erro ocorreu — nível de programa (declarações de topo), nível de membro de classe (campos, métodos, construtores) e nível de instrução (dentro de um bloco). Em cada um destes pontos, o parser regista o erro, avança tokens até encontrar um ponto de sincronização seguro (tipicamente um `;` ou um `}`), e insere um nó de erro dedicado (`ErrorStatement` ou `ErrorMember`) na posição da AST onde a construção falhou, permitindo que o resto do ficheiro continue a ser analisado — e, mais importante para este exame, que a fase semântica possa correr sobre a parte da AST que se manteve válida, em vez de nunca ser executada.



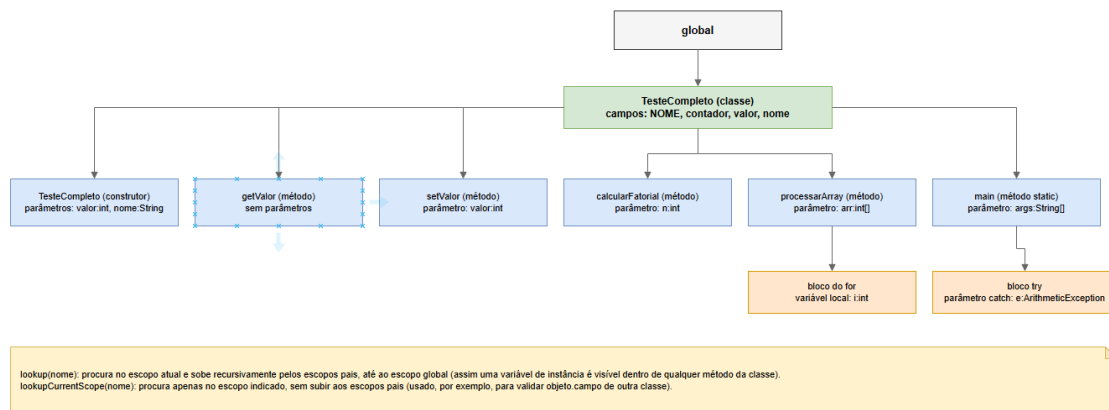
2.4. Construção da Tabela de Símbolos

A Tabela de Símbolos (`SymbolTable`, pacote `com.compiler.symbol`) é construída **durante** a análise sintática — não é uma fase separada — pela razão prática de que é exatamente durante o reconhecimento de uma declaração (de campo, parâmetro, variável local, método, construtor, classe ou interface) que o parser dispõe, sem trabalho adicional, de toda a informação necessária: o nome do identificador, o seu tipo declarado, os modificadores, o valor inicial (se existir), e sobretudo o **escopo atual** em que a declaração ocorre.

A tabela foi desenhada como uma estrutura **hierárquica de escopos**: cada `SymbolTable` mantém uma referência ao seu escopo pai (`parent`) e uma lista dos seus

escopos filhos (children), formando uma árvore que espelha a estrutura de blocos aninhados do programa (escopo global → escopo de classe → escopo de método → escopos de blocos aninhados). Isto permite dois modos de pesquisa distintos, ambos necessários para as verificações semânticas:

- `lookup(nome)` — pesquisa no escopo atual e, se não encontrar, sobe recursivamente pelos escopos pais, até ao escopo global. É este mecanismo que permite que uma variável de instância de uma classe seja visível dentro de qualquer um dos seus métodos, ou que uma variável de um bloco externo seja visível dentro de um bloco `if/while` aninhado.
- `lookupCurrentScope(nome)` — pesquisa apenas no escopo indicado, sem subir aos escopos pais. É usado, por exemplo, para validar o acesso a um membro de **outra** classe (`objeto.metodo()`), onde não faria sentido "vazar" para o escopo de quem está a fazer a chamada.



Cada entrada da tabela (SymbolEntry) guarda exatamente os atributos pedidos pelo enunciado do exame: **nome**, **tipo de dado** (primitivo ou definido pelo utilizador), **endereço de memória** (simulado, atribuído de forma incremental a partir de 0x1000 por escopo, para fins didáticos), **tamanho em bytes** (derivado do tipo — por exemplo `int` = 4 bytes, `double` = 8 bytes, qualquer tipo referência = 8 bytes), **escopo** (caminho completo, ex.: `MinhaClasse.metodo`), **valor de atribuição** (quando existe inicializador), **dimensão** (número de `[]` do tipo, para arrays), além da categoria do símbolo (classe, interface, campo, campo estático, parâmetro, variável local, método ou construtor) e dos seus modificadores (`public`, `static`, `final`, etc.).

Quando o parser tenta inserir um símbolo cujo nome já existe no escopo atual, a inserção **não gera imediatamente um erro** — é registada numa lista separada de declarações duplicadas (`Parser.DuplicateDeclaration`), que é entregue ao analisador semântico. Esta decisão de desenho separa claramente a responsabilidade de **construir** a tabela (parser) da responsabilidade de **decidir como reportar** uma violação de regra semântica (analisador semântico) — sendo esta segunda responsabilidade o verdadeiro objeto de avaliação deste exame.

2.5. Construção do Analisador Semântico

Esta é a componente central e o objeto de avaliação principal deste exame. O Analisador Semântico (`SemanticAnalyzer`, pacote `com.compiler.semantic`) recebe, como entrada, a raiz da Árvore Sintática Abstrata produzida pelo parser (`Program`) e a lista de declarações duplicadas já detetadas durante o parsing, e percorre recursivamente toda a árvore, usando a **Tabela de Símbolos** já construída (acessível a partir de cada nó de classe/método através do respetivo escopo) como fonte de verdade sobre cada identificador.

2.5.1. Metodologia: atribuição de tipo sintetizado

A técnica usada para a verificação de tipos é a clássica **atribuição de tipo sintetizado** (*synthesized type attribute*), típica de analisadores semânticos dirigidos por sintaxe: cada nó de **expressão** da AST, quando visitado pelo método `visitExpression`, devolve um valor `Type` que representa o tipo calculado dessa expressão, calculado a partir dos tipos (já calculados) dos seus nós filhos. Por exemplo, o tipo de uma expressão binária `a + b` é calculado a partir do tipo de `a` e do tipo de `b`, seguindo as regras de promoção numérica do Java; o tipo de uma chamada de método é o tipo de retorno declarado desse método na Tabela de Símbolos; o tipo de um acesso a array é o tipo do array com uma dimensão a menos.

Para representar esses tipos de forma uniforme e comparável, foi construída uma classe auxiliar `Type`, que combina exatamente a mesma informação já presente numa `SymbolEntry` (tipo base primitivo/OBJECT, nome de classe personalizado, número de dimensões de array) com predicados de conveniência (`isNumeric()`, `isBoolean()`, `isString()`, `isArrayType()`, `isReference()` etc.) e, sobretudo, com o método `isAssignableTo(Type alvo)`, que centraliza **todas** as regras de compatibilidade de tipos usadas pelas restantes verificações: promoção numérica (*widening*: `byte`→`short`/`char`→`int`→`long`→ `float`→`double`), aceitação de `null` para qualquer tipo referência, `Object` como super-tipo universal de qualquer referência, e exigência de igualdade estrita de classe/dimensão para arrays e tipos definidos pelo utilizador. Centralizar esta lógica numa única classe evitou duplicar regras de compatibilidade em cada um dos pontos de verificação.

Um segundo aspeto de desenho relevante é o uso de um tipo sentinela, `Type.ERROR`, devolvido sempre que uma sub-expressão já continha um erro. Qualquer verificação de compatibilidade envolvendo `Type.ERROR` é automaticamente considerada válida, o que evita que um único erro (por exemplo, o uso de uma variável não declarada) gere uma cascata de erros secundários derivados do mesmo problema, quando essa expressão é usada dentro de uma expressão maior.

2.5.2. As seis verificações exigidas pelo enunciado

Cada uma das alíneas do enunciado corresponde a um mecanismo concreto e identificável na implementação:

a) Verificação de variável não declarada. Sempre que a AST contém uma referência a um identificador (`IdentifierExpression`), o analisador semântico consulta a Tabela de Símbolos do escopo corrente através de `scope.lookup(nome)`. Se a pesquisa não encontrar nenhuma entrada, e o nome também não corresponder a uma classe conhecida do programa ou da biblioteca padrão, é reportado o erro "Variável 'x' não declarada", na linha e coluna exatas onde o identificador aparece.

b) Verificação de variável declarada duas vezes no mesmo escopo. Como descrito na seção 2.4, o parser já deteta, ao tentar inserir um símbolo cujo nome colide com outro já existente no mesmo escopo, que se trata de uma duplicata. O analisador semântico converte cada uma dessas ocorrências num erro "Símbolo 'x' já declarado no escopo 'Y'", centralizando assim, na fase semântica, a decisão de **reportar** a violação (ainda que a **deteção** técnica já tenha ocorrido durante o parsing, por ser o momento natural para tal).

c) Verificação de incompatibilidade de tipos de dados. Aplicada a todos os operadores binários (aritméticos, relacionais, de igualdade, lógicos e bit a bit) e unários (`!`, `~`, `+`, `-`, `++`, `--`). Cada categoria de operador tem uma regra de compatibilidade própria — por exemplo, os operadores aritméticos exigem que ambos os operandos sejam numéricos (com exceção da concatenação de `String` através do `+`); os operadores lógicos exigem que ambos sejam `boolean`; os operadores relacionais exigem operandos numéricos. Qualquer violação produz um erro do tipo "Operador '+' não pode ser aplicado a tipos incompatíveis 'boolean' e 'int'".

d) Verificação de tipos nos argumentos das funções (métodos). Sempre que a AST contém uma chamada de método (`MethodInvocationExpression`) ou uma criação de objeto (`NewExpression`), o analisador semântico localiza a declaração do método/construtor correspondente na Tabela de Símbolos da classe-alvo, recolhe a lista de parâmetros declarados (a partir do escopo filho correspondente ao método), e compara, um a um, o tipo de cada argumento fornecido com o tipo do parâmetro correspondente — reportando tanto divergências de tipo ("Argumento 2 de 'soma' tem tipo 'String', esperado 'int'") como divergências no número de argumentos ("Número de argumentos incorreto para 'soma': esperado 2, encontrado 1").

e) Verificação de atribuição de um tipo incompatível para outra variável. Duas situações são cobertas: a declaração de uma variável (local ou campo) com inicializador, e a instrução/expressão de atribuição (`=` e as suas variantes compostas `+=`, `-=`, `*=`, etc.). Em ambos os casos, o tipo do valor à direita é comparado, via `Type.isAssignableTo`, com o tipo declarado da variável/campo à esquerda, sendo reportado um erro do tipo "Não é possível atribuir um valor do tipo 'String' para a variável 'z' do tipo 'int'" sempre que a atribuição viole as regras de compatibilidade.

f) Verificação dos argumentos nas estruturas de controlo. Aplicada uniformemente a `if`, `while`, `do-while`, `for` (na sua condição) e `assert`, exigindo que a expressão de condição seja do tipo `boolean`; ao `switch`, exigindo que a expressão testada seja `int/char/String` e que cada rótulo `case` seja de um tipo compatível com essa expressão;

e ao `for-each`, exigindo que o tipo do elemento do array percorrido seja compatível com o tipo declarado da variável do laço.

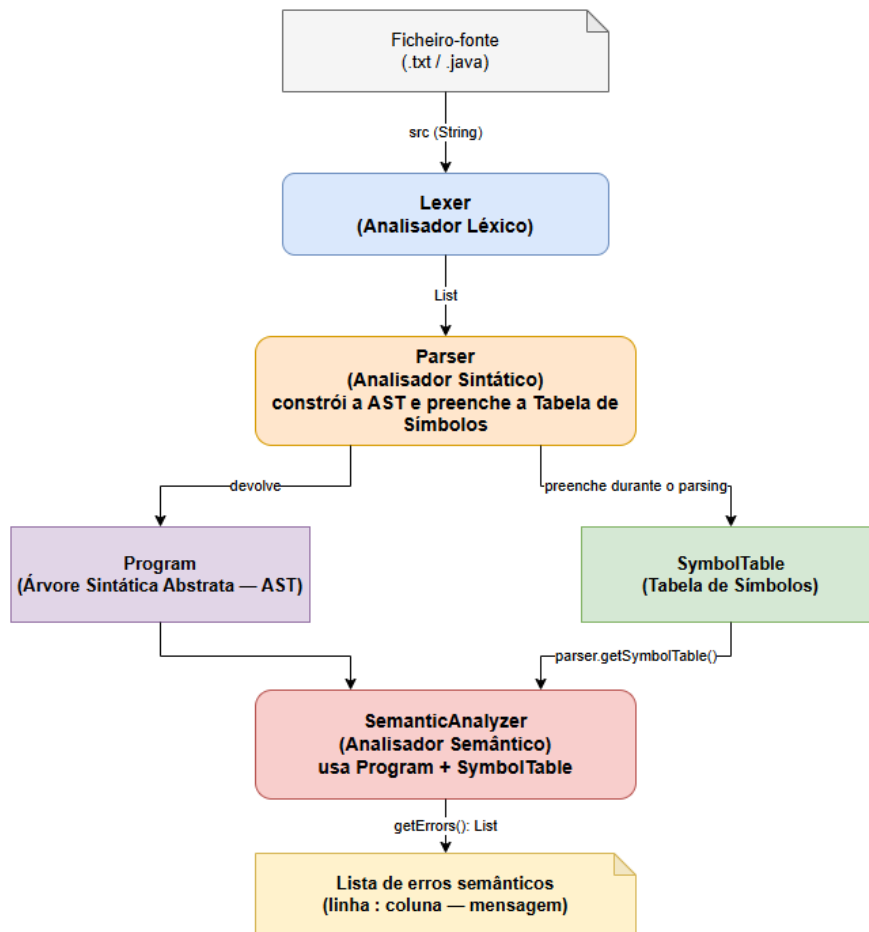
Além destas seis verificações mínimas, o analisador semântico implementado cobre ainda, pelo mesmo mecanismo de tipo sintetizado, a verificação do tipo de retorno de métodos (incluindo a detecção de um `return` com valor num método `void`, ou sem valor num método que devolve um tipo), a compatibilidade de tipos nos dois ramos do operador ternário, a validação da existência de campos e métodos antes de lhes aceder, e a validação do tipo do índice de acesso a arrays e das dimensões de criação de novos arrays.

2.5.3. Limitações conscientes

Por se tratar de um mini-compilador com âmbito académico delimitado, três simplificações foram assumidas e documentadas explicitamente no código: (i) não há resolução de sobrecarga de métodos ou construtores — a Tabela de Símbolos modela "um nome → um símbolo por escopo", pelo que dois métodos com o mesmo nome no mesmo escopo são tratados como uma duplicata, o que é coerente com a alínea (b) do próprio enunciado; (ii) não há modelação de herança entre classes definidas pelo utilizador, sendo apenas `Object` tratado como super-tipo universal de qualquer tipo referência; e (iii) um conjunto fixo de classes da biblioteca padrão de Java (`System`, `String`, `ArrayList`, `Math`, etc.) é aceite através de uma lista interna, uma vez que este compilador não modela o mecanismo de `import` nem o `classpath` do JDK — o objetivo do exercício é validar o código escrito pelo utilizador, e não recompilar a biblioteca padrão de Java.

2.6. Integração das três fases e exemplo de execução completa

As três fases foram integradas num único ponto de entrada, `com.compiler.main.Main`, que executa sequencialmente: leitura do ficheiro-fonte, análise léxica (produzindo a lista de tokens consumida internamente pelo parser), análise sintática (produzindo a AST e a Tabela de Símbolos) e análise semântica (produzindo a lista de erros semânticos), imprimindo no final um relatório focado no que este exame exige demonstrar: o resultado do parser, a Tabela de Símbolos completa (com todos os atributos pedidos pelo enunciado — endereço, tipo, dimensão, valor, escopo, tamanho em bytes) e os erros sintáticos e semânticos (se existirem). A reimpressão do código-fonte lido e da listagem token a token do `Lexer` — que não fazem parte do que o Analisador Semântico precisa de demonstrar — foi deliberadamente desativada em `Main.java`, mantendo-se apenas comentada no código-fonte para eventual depuração isolada da fase léxica.



A título de ilustração, o seguinte excerto de código, colocado num ficheiro de teste dedicado (teste_semantico.txt), foi construído propositadamente para exercitar, num só programa, as seis verificações exigidas pelo enunciado:

```

public class TesteSemantico {
    private int total = 0;
    private int total = 5;           // (b) declaração duplicada

    public int soma(int a, int b) {
        return a + b;
    }

    public void executar() {
        int x = 10;
        String texto = "ola";

        int y = x + naoDeclarada;    // (a) variável não declarada

        int z = texto;               // (c)/(e) tipos incompatíveis

        int resultado = soma(1, "dois"); // (d) argumento de tipo errado
    }
}
  
```

```
if (x) { ... }           // (f) condição não booleana
while (texto) { ... }    // (f) condição não booleana
for (int i = 0; texto; i++) { ... } // (f) condição não booleana
}
}
```

A execução do MINI-COMPILADOR sobre este ficheiro produz exatamente sete erros semânticos, um por cada violação intencionalmente introduzida, confirmando que as seis alíneas do enunciado estão cobertas de forma consistente e verificável (a transcrição completa da saída encontra-se no Apêndice B — Manual do Utilizador, secção 8.2).

3. CONCLUSÃO

O trabalho desenvolvido cumpriu integralmente os objetivos definidos no enunciado do Exame Normal de Compiladores: a construção de um MINI-COMPILADOR completo, cobrindo as três fases do front-end — análise léxica, análise sintática e análise semântica — para um subconjunto realista da linguagem Java.

Do ponto de vista da análise léxica, o exercício permitiu aplicar de forma direta a teoria de autómatos finitos determinísticos, construindo manualmente um autômato de dezenas de estados capaz de reconhecer todas as categorias de token de uma linguagem real, incluindo casos que exigem cuidado adicional, como comentários de bloco aninhados e sequências de escape em literais de texto.

Do ponto de vista da análise sintática, o exercício exigiu confrontar diretamente os dois problemas clássicos indicados no enunciado — ambiguidade e recursividade à esquerda — e resolvê-los através das técnicas padrão de reescrita de gramáticas (introdução de uma produção auxiliar para o "dangling else"; conversão de recursividade à esquerda em iteração para toda a hierarquia de expressões), permitindo a construção de um parser descendente recursivo eficiente, acompanhado de um mecanismo de recuperação de erros em modo pânico que garante que um único erro sintático não impede a análise do resto do programa.

Do ponto de vista da análise semântica — o objeto central deste exame —, o trabalho demonstra que a informação recolhida na Tabela de Símbolos durante o parsing (tipo, escopo, endereço, dimensão, valor inicial de cada símbolo) é suficiente e adequada para implementar, de forma sistemática e reutilizável, todas as verificações semânticas exigidas: detecção de variáveis não declaradas, detecção de declarações duplicadas no mesmo escopo, verificação de compatibilidade de tipos em operadores e atribuições, verificação de tipos e número de argumentos em chamadas de métodos e construtores, e verificação do tipo das condições em todas as estruturas de controlo de fluxo da linguagem. A representação de tipos centralizada na classe `Type`, com um único ponto de decisão sobre compatibilidade de atribuição (`isAssignableTo`), mostrou-se uma escolha de desenho eficaz para evitar duplicação de regras e para garantir consistência entre as diferentes verificações.

Como trabalho futuro, ficam identificadas de forma explícita as simplificações assumidas — ausência de sobrecarga de métodos, ausência de modelação de herança entre classes do utilizador, e uma lista fixa de classes da biblioteca padrão — que constituiriam as extensões naturais deste MINI-COMPILADOR caso o seu âmbito fosse alargado para além do exigido neste exame, nomeadamente rumo a uma fase de geração de código.

4. REFERÊNCIAS BIBLIOGRÁFICAS

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. Compilers: Principles, Techniques, and Tools. 2ª edição. Boston: Pearson/Addison Wesley, 2006. (Conhecido como "The Dragon Book" — referência clássica para autômatos finitos na análise léxica, gramáticas LL e eliminação de recursividade à esquerda, análise semântica dirigida por sintaxe e tabelas de símbolos.)

LOUDEN, Kenneth C. Compiler Construction: Principles and Practice. Boston: PWS Publishing Company, 1997. (Referência para a construção prática de analisadores léxicos e sintáticos por descida recursiva, e para o desenho de tabelas de símbolos hierárquicas por escopo.)

APPEL, Andrew W. Modern Compiler Implementation in Java. 2ª edição. Cambridge: Cambridge University Press, 2002. (Referência para a implementação de compiladores em Java, incluindo desenho de AST e de analisadores semânticos dirigidos por tipos.)

GOSLING, James; JOY, Bill; STEELE, Guy; BRACHA, Gilad; BUCKLEY, Alex; SMITH, Daniel; BIERMAN, Gavin. The Java Language Specification, edição correspondente ao Java SE 17. Oracle America, Inc. Disponível em: docs.oracle.com/javase/17/specs/. (Especificação de referência da gramática e das regras de compatibilidade de tipos — widening, promoção numérica — usadas para validar o subconjunto de Java implementado.)

PARR, Terence. Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages. Raleigh: The Pragmatic Bookshelf, 2010. (Referência para padrões de implementação de parsers descendentes recursivos e de recuperação de erros em modo pânico.)

Material de apoio e slides da disciplina de Compiladores (elaboração da gramática, eliminação de ambiguidade do dangling else, eliminação de recursividade à esquerda, e especificação das verificações semânticas mínimas exigidas para o MINI-COMPILADOR).

5. APÊNDICES

Os apêndices seguintes reproduzem, na íntegra, o Manual do Programador e o Manual do Utilizador do MINI-COMPILADOR, entregues também como documentos autónomos junto com este relatório.

Apêndice A — Manual do Programador

Manual do Programador

MINI-COMPILADOR para um subconjunto da linguagem Java

1. INTRODUÇÃO AO MANUAL

Este manual destina-se a quem vai **ler, manter ou estender** o código-fonte do MINI-COMPILADOR. Ele não explica como *usar* o compilador do ponto de vista de quem só quer analisar um ficheiro `.txt/.java` — isso é o papel do *Manual do Utilizador*, em anexo. Aqui o público-alvo é o próprio autor (ou qualquer outro programador) que precise de entender a arquitetura interna, localizar uma classe responsável por um determinado comportamento, ou adicionar uma nova verificação ao analisador semântico.

O projeto está organizado como um **projeto NetBeans (Ant)** dentro da pasta `Compiler/`, com o código-fonte Java em `Compiler/src/com/compiler/`. O compilador implementa as três fases clássicas do *front-end* de um compilador:

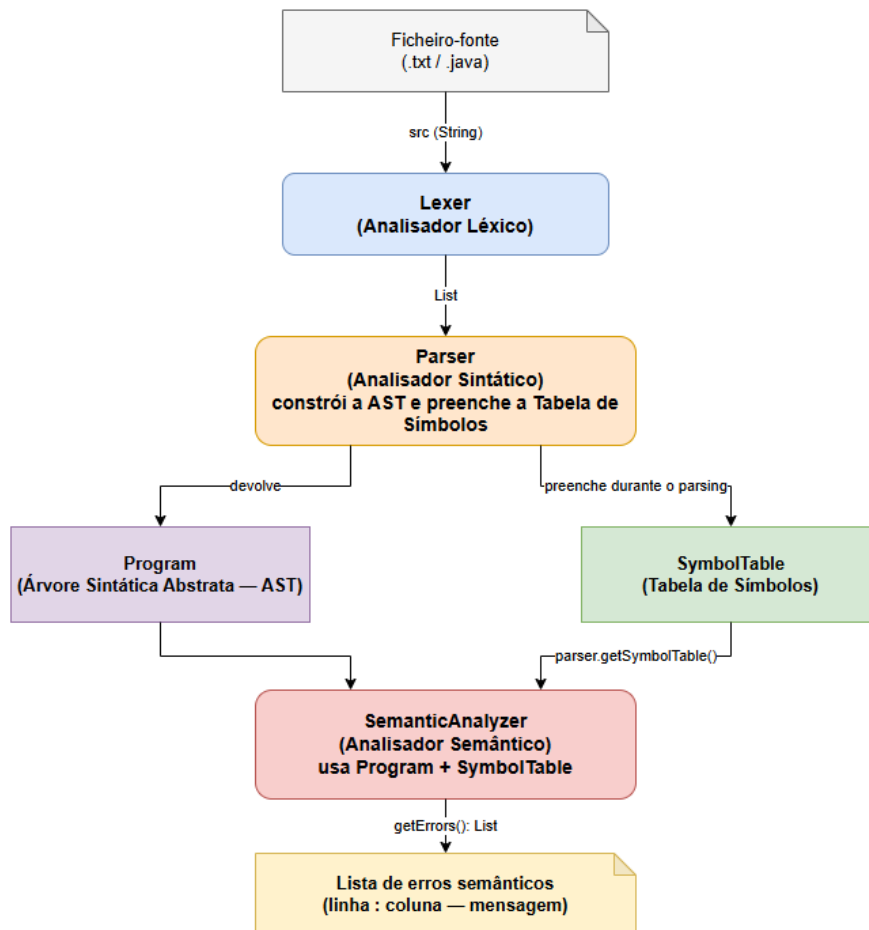
1. **Análise Léxica** (`com.compiler.lexer`) — transforma o texto-fonte numa sequência de *tokens*.
2. **Análise Sintática** (`com.compiler.parser`) — transforma a sequência de tokens numa **Árvore Sintática Abstrata (AST)** e, simultaneamente, preenche a **Tabela de Símbolos** (`com.compiler.symbol`).
3. **Análise Semântica** (`com.compiler.semantic`) — percorre a AST validando as regras semânticas da linguagem, usando a Tabela de Símbolos como fonte de informação sobre cada identificador.

O ponto de entrada de todo o pipeline é a classe `com.compiler.main.Main`.

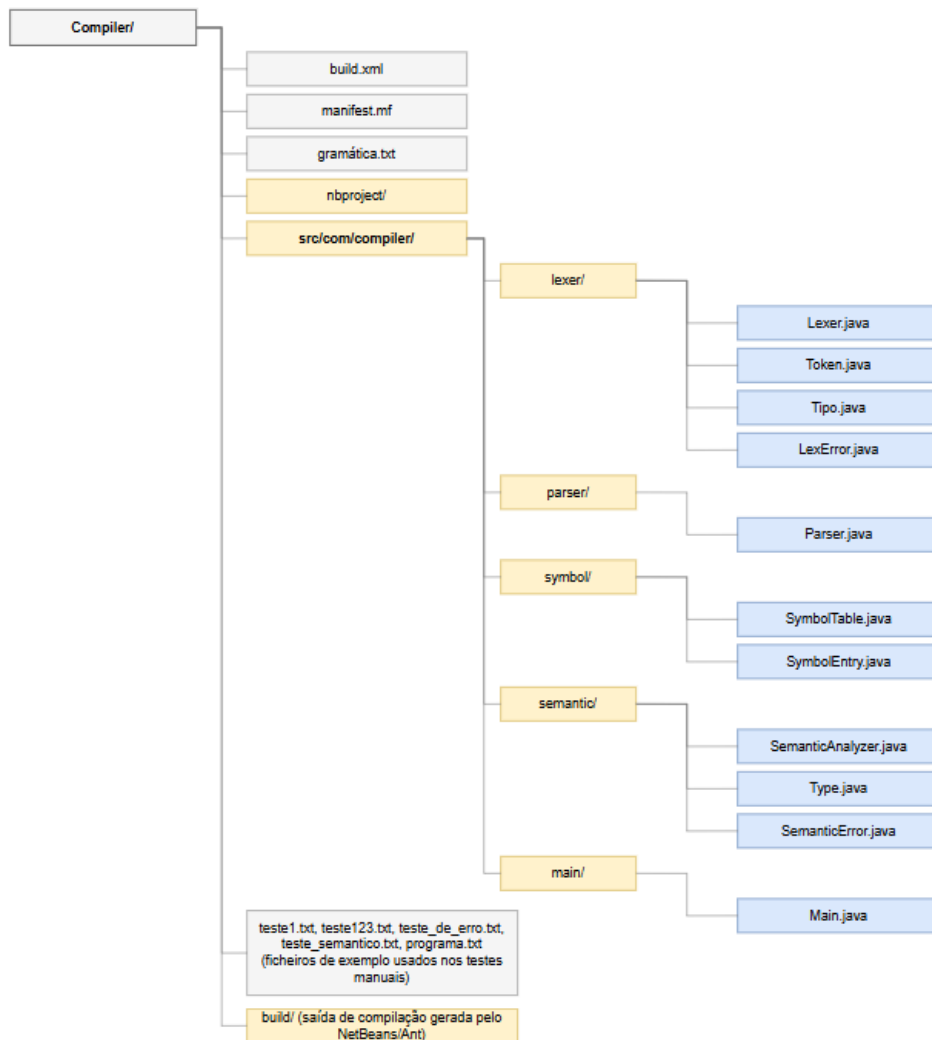
2. VISÃO GERAL DA ARQUITETURA

Um aspeto de desenho importante: **a Tabela de Símbolos é construída pelo `Parser`, não pelo `SemanticAnalyzer`**. O parser já conhece o escopo em que está (classe, método, bloco) no momento em que reconhece uma declaração, por isso é o local mais natural para registar nome, tipo, endereço de memória, tamanho em bytes, escopo e valor inicial de cada símbolo. O SemanticAnalyzer **consome** essa tabela pronta (via `parser.getSymbolTable()`) em vez de a reconstruir, e usa-a para resolver identificadores, validar tipos e detetar declarações duplicadas (a lista de duplicados também é recolhida pelo parser, em `parser.getDuplicateDeclarations()`).

Esta separação de responsabilidades é deliberada e está refletida no Javadoc de SemanticAnalyzer:



3. ORGANIZAÇÃO DO CÓDIGO-FONTE



Não existem dependências externas: todo o compilador usa apenas a biblioteca padrão do Java (`java.util`, `java.nio.file`).

4. O PACOTE LEXER — ANALISADOR LÉXICO

4.1. Tipo (enum)

Define todas as categorias de token reconhecidas: literais (`LITERAL_INTEIRO`, `LITERAL_LONG`, `LITERAL_FLOAT`, `LITERAL_DOUBLE`, `LITERAL_TEXTO`, `LITERAL_CARACTERE`, `LITERAL_ESPECIAL` para `true/false/null`), identificadores, palavras reservadas, todos os operadores (aritméticos, relacionais, lógicos, bit a bit, de atribuição composta, shifts), delimitadores (`( ) { } [ ] ; , . :`), EOF e `DESCONHECIDO` (para caracteres não reconhecidos).

4.2. Token (classe)

Estrutura simples e imutável do ponto de vista de uso: `tipo` (um `Tipo`), `lexema` (o texto original), `linha` e `col` (posição no ficheiro-fonte, usada depois pelo parser e pelo analisador semântico para reportar erros com localização exata).

4.3. LexError (exceção)

`RuntimeException` lançada nos três casos em que o scanner não consegue recuperar sozinho: string não fechada, literal de caractere não fechado e comentário de bloco não fechado. Estes casos abortam a análise porque não há um ponto de sincronização seguro dentro do próprio token.

4.4. Lexer (classe principal)

Metodologia: o scanner é implementado como um **Autómato Finito Determinístico (AFD) explícito**, simulado por uma tabela de transições (`transicao(Estado, char): Estado`) sobre um enum de 50+ estados (Q0 a Q63). Cada chamada a `scan()` percorre o código-fonte carácter a carácter, mantendo:

- `estadoAtual` — o estado corrente do autómato;
- `lexemaBuffer` — o lexema acumulado até ao momento;
- `pos`, `linha`, `col` — posição de leitura no ficheiro-fonte.

O ciclo principal (`processarToken`) só faz uma transição de estado quando o próximo carácter é compatível com a categoria de token que já está a ser reconhecida; assim que a transição não é possível (`transicao(...) == null`), o lexema acumulado é fechado (`finalizarToken`) e classificado.

Casos especiais tratados fora da tabela genérica de transições, por simplicidade e clareza:

Operadores compostos (`processarOperadorEspecifico`): em vez de representar todas as combinações de 1 a 4 caracteres (`=`, `==`, `<`, `<=`, `<<`, `<<=`, `>>=`, etc.) apenas na tabela de estados, o lexer resolve diretamente por *lookahead* de 1–3 caracteres a partir do primeiro carácter do operador. Isto simplifica bastante o autómato genérico sem perder o determinismo.

Comentários de linha (`/\`) e de bloco (`/\* ... */`, com suporte a aninhamento): tratados por sub-rotinas dedicadas (`pularComentarioLinha`, `pularComentarioBloco`) que **não geram token** — são simplesmente ignorados, avançando `pos/linha/col`.

Strings (`processarString`) e **caracteres** (`processarChar`): tratados por sub-rotinas dedicadas devido à necessidade de reconhecer sequências de escape (`\`, `\\`, `\n`, etc.) sem as interpretar (`\\` seguido de qualquer carácter é aceite literalmente).

Números (`processarNumero`): a tabela de estados distingue inteiro (Q1), sufixo `L/l` (`long`, Q2), ponto decimal (Q3→Q4) e sufixo `F/f/D/d` (Q5), decidindo entre `LITERAL_INTEIRO`, `LITERAL_LONG`, `LITERAL_FLOAT` e `LITERAL_DOUBLE` no fecho do token.

Identificadores e palavras reservadas (`processarIdentificador`): a tabela reconhece sequências de letras/dígitos/`_/$` (estado Q8); no fecho do token, `finalizarToken`

consulta as listas `keywordsArray` (50 palavras reservadas de Java) e `literalsArray` (`true`, `false`, `null`) para decidir entre `PALAVRA_RESERVADA`, `LITERAL_ESPECIAL` ou `IDENTIFICADOR`.

Erros léxicos: caracteres não reconhecidos (`Tipo.DESCONHECIDO`) não abortam a análise — são reportados em `System.err` como aviso e ignorados, permitindo que o scanner continue (política de recuperação "descartar e continuar"). Já strings/chars/comentários de bloco não fechados lançam `LexError`, que se propaga até `Main` (não há recuperação parcial dentro de um literal inacabado).

5. O PACOTE `PARSER` — ANALISADOR SINTÁTICO E AST

`Parser.java` é o ficheiro mais extenso do projeto (~2800 linhas) porque concentra três responsabilidades:

1. O **parser descendente recursivo** propriamente dito (reconhecimento da gramática);
2. A **definição da AST** (todas as classes `Node/Statement/Expression` estão aninhadas dentro de `Parser`, como *static nested classes*);
3. O **preenchimento da Tabela de Símbolos** durante o reconhecimento de declarações.

5.1. Metodologia de construção da gramática

A gramática usada (ver `gramática.txt` na raiz do projeto `Compiler/`) foi preparada especificamente para permitir um parser **LL(1) / descendente recursivo com lookahead limitado**, eliminando os dois problemas clássicos que impediriam essa técnica:

Ambiguidade do "dangling else": resolvida seguindo a convenção-padrão (o `else` liga-se sempre ao `if` mais próximo sem `else`), através da regra `<instrucaolf> ::= "if" "(" <expressao> ")" <instrucaoNoElse> ("else" <instrucao>)?`, em que `<instrucaoNoElse>` proíbe explicitamente um `if` "aberto" antes de um `else` do próprio ramo — implementado no código em `parseStatement/parseSelfStatement` fazendo o `else` associar-se sempre à chamada recursiva mais interna.

Recursividade à esquerda nas expressões: a gramática de expressões da linguagem Java é naturalmente recursiva à esquerda (ex.: `expressaoAditiva ::= expressaoAditiva ("+"|" -") expressaoMultiplicativa | expressaoMultiplicativa`). Essa recursividade foi eliminada reescrevendo cada nível de precedência como um **laço iterativo** sobre a produção equivalente à direita: `expressaoAditiva ::= expressaoMultiplicativa (("+"|" -") expressaoMultiplicativa)*`. Isto é o que se vê em métodos como `parseAdditive()`, `parseMultiplicative()`, `parseShift()`, etc. — todos seguem o padrão "parse do nível abaixo, depois while consumindo o operador do próprio nível e o operando seguinte".

Esta reescrita produz uma **hierarquia de precedência** (do menor para o maior), cada um implementado como um método próprio, chamando sempre o nível imediatamente abaixo:

Esta é a mesma ideia por trás de um *Pratt parser*/parser de precedência de operadores, mas implementada de forma totalmente explícita através da hierarquia de métodos acima (sem tabela de precedência em runtime), o que facilita muito a leitura e a depuração.

Cada método chama o nível seguinte e, num laço, consome o(s) operador(es) do seu próprio nível — eliminando a recursividade à esquerda



5.2. A Árvore Sintática Abstrata (AST)

Todas as classes de nó (Program, TypeDeclaration, ClassDeclaration, InterfaceDeclaration, MethodDeclaration, ConstructorDeclaration, FieldDeclaration, Statement e as suas ~20 subclasses, Expression e as suas ~25 subclasses) estendem a classe base Node, que carrega line/col para permitir localizar qualquer nó no ficheiro-fonte original (usado depois pelo analisador semântico para relatar erros). A associação

Token → Node é feita através dos utilitários `mark(node, token)` e `markPos(node, line, col)`.

Nós de erro dedicados (`ErrorStatement`, `ErrorMember`) permitem que o parser **recupere de um erro sintático sem parar completamente**: em vez de abortar, o parser insere um nó de erro no lugar da construção que falhou e continua a processar o resto do ficheiro (ver secção 13).

5.3. Preenchimento da Tabela de Símbolos

Sempre que o parser reconhece uma declaração (campo, parâmetro, variável local, método, construtor, classe, interface), chama `addToSymbolTable(name, kind, typeName, arrayDepth, initialValue, modifiers)`. Este método:

1. Resolve o `DataType` (primitivo, `String`, `void` ou `OBJECT` com `customTypeName`) e o respetivo tamanho em bytes (`DataType.getSize()`).
2. Calcula um **endereço de memória simulado**, através de `currentScope.allocateAddress(size)` (endereços incrementais a partir de `0x1000`, apenas para fins didáticos — não correspondem a memória real).
3. Cria uma `SymbolEntry` com todos os atributos: nome, tipo, tipo personalizado, tamanho, endereço, caminho do escopo, valor inicial, modificadores, dimensão de array, linha e coluna.
4. Tenta inserir no `currentScope` (`SymbolTable.addSymbol`); se já existir um símbolo com o mesmo nome nesse escopo, a inserção falha e a duplicata é registada em `duplicateDeclarations` — **não é lançado um erro sintático**; a decisão de como reportar isso é da responsabilidade do analisador semântico (verificação **b**, secção 7).

Cada vez que o parser entra num novo escopo (corpo de classe, de método, de construtor, de bloco `{ }`), cria uma nova `SymbolTable` filha (`new SymbolTable(currentScope, nomeDoEscopo)`) e restaura o escopo anterior ao sair (padrão `save/restore` visível em `visitMethodDeclaration`, `parseBlock`, etc. — o mesmo padrão é usado no `SemanticAnalyzer` para `currentClassName/currentClassScope/currentReturnType`).

5.4. Alocação recursiva de sub-escopos

Note-se que um escopo filho herda o próximo endereço livre do escopo pai (`this.nextMemoryAddress = parent.nextMemoryAddress` no construtor de `SymbolTable`) — isto é uma simplificação didática que evita colisões de endereço entre variáveis de escopos diferentes, sem modelar de forma precisa uma pilha de ativação real.

6. O PACOTE SYMBOL — TABELA DE SÍMBOLOS

6.1. SymbolEntry

Guarda exatamente os atributos exigidos pelo enunciado para cada identificador:

Atributo	Campo Java	Descrição
----------	------------	-----------

Nome	<code>name</code>	Identificador declarado
Tipo de dado	<code>dataType / customTypeName</code>	Primitivo (<code>DataType</code>) ou tipo definido pelo utilizador
Endereço de memória	<code>memoryAddress</code>	Endereço simbólico atribuído pelo parser
Tamanho em bytes	<code>sizeInBytes</code>	Deriva de <code>DataType.getSize()</code> (ex.: <code>int=4</code> , <code>double=8</code> , referência=8)
Escopo	<code>scope</code>	Caminho completo do escopo (<code>Classe.metodo</code>)
Valor de atribuição	<code>initialValue</code>	Valor/expressão do inicializador, se existir
Dimensão	<code>arrayDimensions</code>	Número de <code>[]</code> do tipo
Categoria (<code>kind</code>)	<code>kind</code>	<code>CLASS</code> , <code>INTERFACE</code> , <code>FIELD</code> , <code>STATIC_FIELD</code> , <code>PARAMETER</code> , <code>LOCAL_VARIABLE</code> , <code>METHOD</code> , <code>CONSTRUCTOR</code>
Modificadores	<code>modifiers</code>	<code>public</code> , <code>static</code> , <code>final</code> , etc.
Posição	<code>line, column</code>	Localização da declaração

Também expõe utilitários: `getFullName()` (nome do tipo com sufixos `[]`), `isPrimitive()/isReference()` e `getMemoryRegion()` (mapeamento didático para "Stack", "Heap" ou "Área de Classe", consoante o `kind`).

6.2. SymbolTable

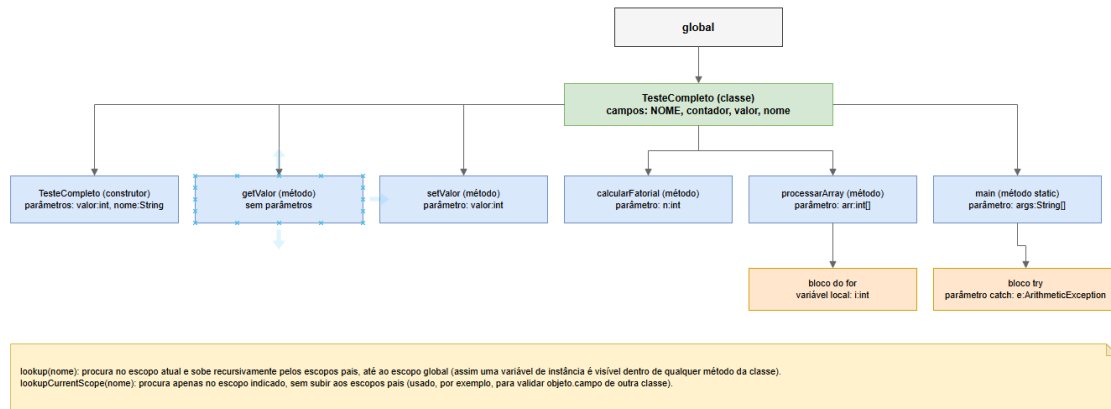
Implementa uma tabela de símbolos **hierárquica** (uma árvore de escopos): cada `SymbolTable` tem um `parent` opcional e uma lista de `children`. As operações principais:

`addSymbol(entry)` — insere no escopo atual; devolve `false` em caso de colisão de nome (usado para detetar duplicados).

`lookup(name)` — procura no escopo atual e, recursivamente, nos escopos ascendentes (`parent`) até ao escopo global — é assim que uma variável declarada numa classe é visível dentro dos seus métodos, e que uma variável de um bloco externo é visível num bloco aninhado.

`lookupCurrentScope(name)` — procura **apenas** no escopo atual, sem subir aos pais; usado para validar membros de uma classe específica (ex.: acesso `objeto.campo`) sem "vazar" para o escopo de quem está a chamar.

`printFullTable()` — imprime a tabela de símbolos completa e formatada (usada por `Main` para o relatório de compilação)



7. O PACOTE SEMANTIC — ANALISADOR SEMÂNTICO

Esta é a componente central deste exame. A classe `SemanticAnalyzer` recebe, no construtor, a `Program` (raiz da AST) e a lista de `Parser.DuplicateDeclaration` recolhida pelo parser, e expõe um único método público de análise: `analyze()`.

7.1. Type — representação de tipos

Antes de descrever as verificações, é essencial entender `Type`, a classe que o analisador semântico usa para representar o **tipo de qualquer expressão** calculado durante a travessia da AST. Cada `Type` guarda:

base (um `SymbolEntry.DataType`: `INT`, `DOUBLE`, `BOOLEAN`, `STRING`, `OBJECT`, `VOID`, ...);

className (para tipos referência definidos pelo utilizador);

arrayDepth (número de dimensões de array);

classRef (marca um `Type` que representa uma referência a um *nome de classe*, como em `System.out`, e não um valor);

flags internas `nullType` e `error`.

Os métodos mais importantes:

`isAssignableTo(Type target)` — implementa as regras de **compatibilidade de atribuição** (usadas nas verificações c, d e e): promoção numérica (*widening*) `byte`→`short`/`char`→`int`→`long`→`float`→`double`, aceitação de `null` para qualquer tipo referência, `Object` como super-tipo universal de qualquer referência, e igualdade estrita de `className/arrayDepth` para arrays e classes definidas pelo utilizador.

`numericResult(Type a, Type b)` — determina o tipo resultante de uma operação aritmética binária, seguindo a promoção padrão do Java (se um dos operandos é `double`, o resultado é `double`; senão `float`; senão `long`; senão `int`).

Predicados de conveniência: `isNumeric()`, `isBoolean()`, `isIntegral()`, `isString()`, `isArrayType()`, `isReference()`, `isVoid()`, `isError()`, `isNull()`.

`Type.ERROR` — um tipo sentinela usado para **propagar erros sem os reportar em duplicado**: qualquer operação envolvendo `Type.ERROR` é considerada automaticamente compatível (`isAssignableTo` devolve `true`), evitando uma "cascata"

de erros semânticos derivados de um único erro original (ex.: usar uma variável não declarada numa expressão maior não deve gerar 3 ou 4 erros adicionais).

7.2. Estrutura da travessia

`analyze()` segue esta sequência:

1. `collectClassScopes(...)` — pré-processamento que regista, para cada classe/interface (incluindo aninhadas), o mapeamento `nome da classe` → `SymbolTable` do seu corpo. Isto é necessário porque o acesso a membros de **outra** classe (`objeto.metodo()`, `Classe.campoEstatico`) só pode ser resolvido depois de todas as classes do programa serem conhecidas — independentemente da ordem em que aparecem no ficheiro-fonte.
2. `reportDuplicateDeclarations()` — converte cada `Parser.DuplicateDeclaration` recolhida durante o parsing num `SemanticError` (verificação **b**).
3. Para cada `TypeDeclaration` do programa, `visitTypeDeclaration` percorre recursivamente todos os seus membros (`visitClassMember`), instruções (`visitStatement`) e expressões (`visitExpression`), acumulando `Type` como valor de retorno de `visitExpression` — este é o clássico padrão de **verificação de tipos por atribuição de tipo sintetizado** (o tipo de um nó é calculado a partir dos tipos dos seus filhos).

Durante a travessia, três variáveis de instância mantêm o "contexto atual" (à semelhança do que o `Parser` faz com `currentScope`):

`currentClassName` / `currentClassScope` — para resolver `this`, chamadas de método sem qualificador e campos herdados do escopo da própria classe;

`currentReturnType` — para validar `return` (verificação relacionada a tipos de retorno, incluída na alínea c/e do enunciado).

7.3. Mapeamento direto às alíneas do enunciado

Alínea do enunciado	Onde está implementado	Mensagem de erro típica
a) Variável não declarada	<code>visitIdentifier</code> — <code>scope.lookup(name)</code> devolve <code>null</code> e o nome não é uma classe conhecida	Variável 'x' não declarada
b) Variável declarada duas vezes no mesmo escopo	<code>reportDuplicateDeclarations()</code> , a partir de <code>SymbolTable.addSymbol</code> que devolveu <code>false</code> no parser	Símbolo 'x' já declarado no escopo 'Y'
c) Incompatibilidade de tipos de dados	<code>visitBinary</code> / <code>incompatibleOperands</code> (operadores aritméticos, relacionais, lógicos, bit a bit) e <code>visitUnary/visitPostfix</code>	Operador '+' não pode ser aplicado a tipos incompatíveis 'boolean' e 'int'
d) Tipos dos argumentos de funções/métodos	<code>checkArguments</code> , chamado a partir de <code>visitMethodInvocation</code> e	Argumento 2 de 'soma' tem tipo 'String', esperado 'int'

	visitNew (construtores)	
e) Atribuição de tipo incompatível a outra variável	checkAssignable (declarações com inicializador) e visitAssignment (operador = e atribuições compostas +=, -=, ...)	Não é possível atribuir um valor do tipo 'String' para a variável 'z' do tipo 'int'
f) Argumentos das estruturas de controlo	checkBooleanCondition (if/while/do-while/for/assert), visitForEach (compatibilidade do tipo do elemento com a variável do for-each) e visitSwitch (tipo da expressão e dos case)	Condição de 'if' deve ser do tipo boolean, mas é 'int'

Além destas seis verificações mínimas exigidas, o analisador implementado cobre também: tipo de retorno de métodos (visitReturn), compatibilidade de tipos no operador ternário (visitConditional), existência de campos/métodos antes de aceder a eles (visitFieldAccess, visitMethodInvocation), correspondência de número de argumentos em chamadas e construções new (checkArguments, visitNew), tipo do índice de array (visitArrayAccess), tipo das dimensões de new T[n] (visitNewArray) e compatibilidade de tipo entre o elemento de um array e a variável de um for-each (visitForEach).

7.4. SemanticError

Estrutura mínima — line, col, message — com um método format() que produz a string final "Erro semântico em L:C - mensagem". Todas as mensagens de erro são geradas em português para consistência com o resto da ferramenta.

7.5. Limitações assumidas (documentadas no próprio código)

O Javadoc de SemanticAnalyzer documenta explicitamente três simplificações conscientes, coerentes com o âmbito de um "mini-compilador":

1. **Não há resolução de sobrecarga (overloading)** de métodos/construtores: o parser já trata um segundo método/construtor com o mesmo nome, no mesmo escopo, como um símbolo duplicado (alínea b). Este é um efeito colateral aceite: a tabela de símbolos deste mini-compilador modela "um nome → um símbolo por escopo", tal como pedido no enunciado, e não "um nome → uma lista de assinaturas".
2. **Não há modelagem de herança** entre classes definidas pelo utilizador — apenas Object é tratado como super-tipo universal de qualquer tipo referência (Type.isAssignableTo).
3. **Classes da biblioteca padrão** (System, Math, String, coleções, etc.) são aceites através de uma whitelist fixa (KNOWN_BUILTIN_CLASSES), porque este compilador não modela imports nem o classpath do JDK — o objetivo é validar o código do utilizador, não recompilar a biblioteca padrão de Java.

8. O PACOTE MAIN — PONTO DE ENTRADA

Main.main(String[] args):

1. Lê o caminho do ficheiro-fonte do primeiro argumento da linha de comandos, ou usa "teste_de_erro.txt" por omissão.
2. Corre o Lexer (lexer.scan()) para obter a lista de tokens que o Parser vai consumir — mas **não imprime** o código-fonte nem a lista de tokens por omissão (ver nota abaixo).
3. Cria o Parser com a lista de tokens e chama parseProgram().
4. Se o parser reportou erros sintáticos, imprime-os, mas **continua** a análise na mesma (o parser já recuperou usando modo pânico — ver secção 13), porque nem sempre um erro sintático local invalida o resto do ficheiro.
5. Imprime a tabela de símbolos completa (symbolTable.printFullTable()).
6. Cria o SemanticAnalyzer com a Program e as duplicatas, chama analyze(), e imprime a lista de erros semânticos (se existirem).
7. Produz um resumo final colorido (verde/vermelho via códigos ANSI) com o número total de erros (sintáticos + semânticos).

Nota de desenho: Main.java mantém, comentados (não removidos), os blocos que reimprimiam um banner decorativo, o código-fonte lido e a lista completa de tokens do Lexer. Esses blocos foram desativados porque não fazem parte do que o **enunciado do Analisador Semântico** pede para ser demonstrado: o enunciado exige que o semântico valide a AST com base nos atributos já gravados na Tabela de Símbolos (endereço, tipo, dimensão, valor, escopo, tamanho em bytes) — não que o programa reimprima o código-fonte ou o fluxo de tokens do Lexer (essa é uma preocupação da fase léxica, já coberta noutro momento de avaliação). Ficam apenas comentados, e não apagados, para que continuem disponíveis caso seja necessário depurar o Lexer isoladamente — bastando descomentá-los.

9. FLUXO DE EXECUÇÃO COMPLETO (PASSO A PASSO)

```
1. Main.main(args)
2. src = Files.readString(caminho)
3. tokens = new Lexer(src).scan()
4. parser = new Parser(tokens)
5. program = parser.parseProgram() // preenche symbolTable ao longo do
   caminho
6. symbolTable = parser.getSymbolTable()
7. duplicates = parser.getDuplicateDeclarations()
8. analyzer = new SemanticAnalyzer(program, duplicates)
9. analyzer.analyze()
10. erros = parser.getErrors() + analyzer.getErrors()
11. imprime tokens, tabela de símbolos, erros sintáticos, erros semânticos, resumo
    final
```


10. COMO COMPILAR O PROJETO

Opção A — via NetBeans

Abrir a pasta `Compiler/` como projeto NetBeans (já contém `nbproject/`) e usar **Run** → **Build Project** (ou `Shift+F11`). O `build.xml` (Ant) trata da compilação e da geração do `.jar` com o `Main-Class` correto.

Opção B — via linha de comandos (sem IDE)

A partir da pasta `Compiler/`:

```
# compilar todas as classes para uma pasta de saída
find src -name "*.java" > sources.txt
javac -d build/classes -encoding UTF-8 @sources.txt
```

No Windows/PowerShell, equivalente:

```
Get-ChildItem -Recurse -Filter *.java src | ForEach-Object { $_.FullName } >
sources.txt
javac -d build\classes -encoding UTF-8 "@sources.txt"
```

Requisitos: **JDK 17 ou superior** (o código usa `switch` com expressão de padrão (`switch (kind) { case X -> ...}`), *records* implícitos via `Optional`, e sintaxe de *pattern matching* `instanceof` como em `if (td instanceof ClassDeclaration cd)`).

11. COMO EXECUTAR O PROJETO

Depois de compilado (classes em `build/classes` ou `Compiler/build/classes` se usado o NetBeans):

```
cd build/classes
java com.compiler.main.Main caminho/para/o/ficheiro.txt
```

Se nenhum argumento for passado, o programa tenta abrir `teste_de_erro.txt` no diretório de trabalho atual.

Nota sobre acentuação no terminal Windows: como as mensagens de erro estão em português com acentos, é recomendável correr com a *code page* UTF-8 e forçar a codificação da JVM, para evitar caracteres corrompidos na consola:

```
chcp 65001
java "-Dfile.encoding=UTF-8" "-Dstdout.encoding=UTF-8" com.compiler.main.Main
ficheiro.txt
```

(Ver mais detalhes no *Manual do Utilizador*, secção "Execução".)

12. COMO ESTENDER O COMPILADOR (GUIA PRÁTICO)

Esta secção existe para acelerar tarefas comuns de manutenção.

12.1. Adicionar uma nova verificação semântica

1. Identificar o(s) nó(s) da AST relevante(s) em `Parser.java` (ex.: uma nova instrução ou uma nova forma de expressão).
2. Se ainda não existir, adicionar o caso correspondente em `SemanticAnalyzer.visitStatement` ou `visitExpression`.
3. Implementar a regra usando os predicados já existentes em `Type` (`isNumeric()`, `isAssignableTo(...)`, etc.) — evitar comparar `DataType` diretamente para preservar a lógica de *widening* já centralizada em `Type`.
4. Reportar o erro com `errors.add(new SemanticError(line, col, mensagem))`, reutilizando `positionOf(expr)` para obter a posição mesmo quando a expressão está entre parênteses.
5. Devolver sempre `Type.ERROR` (ou um tipo coerente) do método de verificação, para não propagar erros em cascata.

12.2. Adicionar um novo tipo de instrução (statement) à linguagem

1. Adicionar a regra na gramática (`gramática.txt`).
2. Criar a classe do nó AST como *nested class* de `Parser`, estendendo `Statement`.
3. Implementar o método `parseXYZStatement()` no `Parser`, e ligar a chamada em `parseStatement()` (o método central de despacho por *lookahead* da palavra-chave).
4. Adicionar o caso correspondente em `SemanticAnalyzer.visitStatement`.

12.3. Adicionar suporte a uma nova classe da biblioteca padrão

Adicionar o nome à constante `KNOWN_BUILTIN_CLASSES` em `SemanticAnalyzer.java`. Isto é suficiente para que identificadores como `NomeDaClasse.metodo(...)` deixem de ser reportados como "variável não declarada" — o analisador não valida os membros dessas classes (não faz parte do âmbito do mini-compilador), apenas aceita a referência ao nome.

12.4. Depurar um caso em que a Tabela de Símbolos não tem a entrada esperada

Verificar sempre, pela ordem:

1. Se a declaração foi de facto processada por `addToSymbolTable` no `Parser` (colocar um `System.out.println` temporário, ou `breakpoint`).
2. Se o escopo (`currentScope`) no momento da declaração é o esperado — erros deste tipo aparecem tipicamente quando se esquece de restaurar `currentScope` ao sair de um bloco/método (padrão *save/restore*).
3. Se a procura está a usar `lookup` (sobe aos escopos pai) quando devia usar `lookupCurrentScope` (não sobe), ou vice-versa.

13. TRATAMENTO E RECUPERAÇÃO DE ERROS

13.1. Erros léxicos

Ver secção 4.4: caracteres desconhecidos são ignorados com aviso; strings/chars/comentários de bloco não fechados lançam `LexError` (erro fatal, sem recuperação).

13.2. Erros sintáticos — modo pânico em três níveis

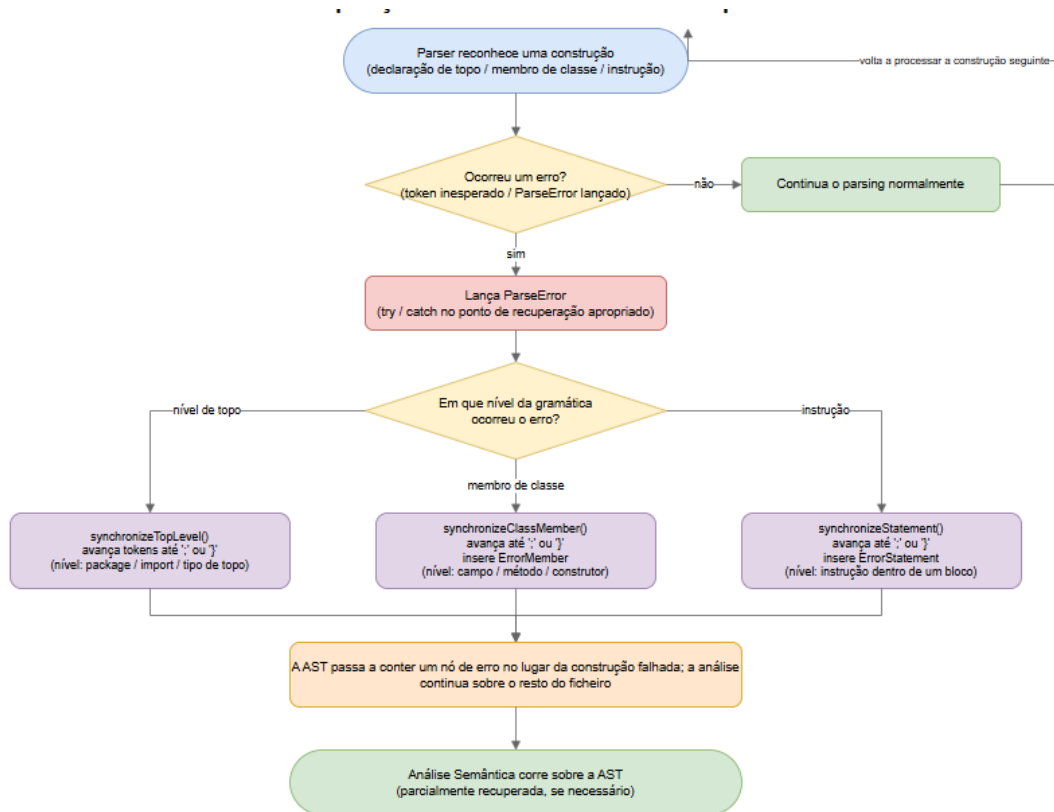
O `Parser` implementa **recuperação de erros em modo pânico** (*panic-mode error recovery*), com três pontos de sincronização independentes, cada um usado no nível da gramática apropriado:

`synchronizeTopLevel()` — usado quando falha o parsing de uma declaração de `package/import`/tipo de topo; avança tokens até ao próximo `;` ou `}`.

`synchronizeClassMember()` — usado quando falha o parsing de um membro de classe (campo, método, construtor); permite que o resto da classe continue a ser analisado mesmo que um membro esteja malformado.

`synchronizeStatement()` — usado quando falha o parsing de uma instrução dentro de um bloco; o erro é isolado à instrução atual.

Cada ponto de recuperação está envolvido num `try { ... } catch (ParseError error) { synchronizeXyz(); }`, e o nó em falta é substituído por um nó de erro (`ErrorStatement/ErrorMember`) para que a AST permaneça estruturalmente válida e a análise semântica possa continuar sobre o resto do ficheiro.



Mensagens de erro sintático especializadas (ex. `errorMissingSemicolon`, `errorMissingOpenBrace`, `errorMissingExpression`) produzem texto mais específico do que um genérico "token inesperado", indicando o contexto (ex.: "esperado ';' após declaração de variável").

13.3. Erros semânticos

O `SemanticAnalyzer` **nunca lança exceções** durante a análise normal — todos os problemas são acumulados na lista `errors` e devolvidos no fim via `getErrors()`. Isto permite reportar **todos** os erros semânticos de um ficheiro numa só execução, em vez de parar no primeiro. O uso sistemático de `Type.ERROR` como valor de retorno para expressões inválidas evita cascatas de erros derivados do mesmo problema original.

14. Convenções de código adotadas

Identificadores de domínio (classes, métodos, variáveis do próprio compilador) e mensagens de erro visíveis ao utilizador estão em **português**, alinhado com o facto de este ser um projeto académico em português.

Cada fase expõe uma API mínima e consistente: `hasErrors()` + `getErrors(): List<String>`. Padrão *save/restore* de contexto (`currentScope`, `currentClassName`, `currentReturnType`) sempre que se entra/sai de um escopo — nunca é feita mutação de contexto sem o correspondente restauro no mesmo método.

Nós da AST são classes aninhadas estáticas de Parser, agrupadas por categoria (Declaration, Statement, Expression) através de uma hierarquia de classes base simples, sem *frameworks* de terceiros.

15. TESTES E FICHEIROS DE EXEMPLO

A raiz de `Compiler/` inclui vários ficheiros de entrada usados como testes manuais de regressão (não há *testes automatizados* com JUnit neste projeto — a verificação é feita correndo o `Main` sobre cada ficheiro e inspecionando a saída):

Ficheiro	Propósito
<code>programa.txt</code>	Programa simples e correto (tipos primitivos, if, comentários)
<code>teste1.txt</code>	Cobre variações de if/else/ternário
<code>teste123.txt</code>	Programa mais completo: construtores, campos estáticos, arrays, try/catch/finally, recursão
<code>teste_de_erro.txt</code>	Ficheiro propositadamente malformado (testa recuperação sintática)
<code>teste_semantico.txt</code>	Ficheiro com as seis classes de erro semântico (a–f) lado a lado, para validar o analisador semântico

Recomenda-se, ao alterar o `SemanticAnalyzer`, correr sempre o compilador sobre `teste_semantico.txt` e confirmar que as sete linhas de erro esperadas continuam a ser reportadas (ver o *Manual do Utilizador*, secção de exemplos, para o conteúdo exato e a saída esperada).

16. LIMITAÇÕES CONHECIDAS E TRABALHO FUTURO

Sem sobrecarga de métodos/construtores (ver 7.5).

Sem modelagem de herança entre classes do utilizador.

Sem verificação de exceções verificadas (*throws*) nem de fluxo de controlo (ex.: "variável pode não ter sido inicializada", "falta um `return` em todos os caminhos").

Sem geração de código (o âmbito deste exame termina na análise semântica — não há back-end).

A tabela de símbolos usa endereços de memória simbólicos incrementais, não uma modelação real de pilha/heap.

Estas limitações são intencionais e proporcionais ao âmbito de um "mini-compilador" académico, sendo aliás as mesmas assumidas e documentadas no próprio Javadoc de `SemanticAnalyzer`.

17. REFERÊNCIA RÁPIDA DE CLASSES (API)

Classe	Pacote	Responsabilidade principal
--------	--------	----------------------------

Lexer	lexer	Converte código-fonte em List<Token>
Token	lexer	Um token: tipo, lexema, linha, coluna
Tipo	lexer	Enum de categorias de token
LexError	lexer	Exceção para erros léxicos fatais
Parser	parser	Parser descendente recursivo; define a AST; preenche a SymbolTable
Parser.Program	parser	Raiz da AST
Parser.DuplicateDeclaration	parser	Registo de um símbolo duplicado detetado durante o parsing
SymbolTable	symbol	Tabela de símbolos hierárquica (uma por escopo)
SymbolEntry	symbol	Atributos de um símbolo (nome, tipo, endereço, escopo, ...)
SemanticAnalyzer	semantic	Percorre a AST e valida as regras semânticas (a-f + extras)
Type	semantic	Representação de tipo usada nas verificações semânticas
SemanticError	semantic	Um erro semântico: linha, coluna, mensagem
Main	main	Ponto de entrada; orquestra as três fases e imprime o relatório

18. CONCLUSÃO

Este manual reuniu toda a informação necessária para que um programador — o próprio autor, num momento futuro, ou qualquer outra pessoa que herde este código — consiga situar-se rapidamente na arquitetura do MINI-COMPILADOR, localizar a classe responsável por um determinado comportamento e alterar ou estender o sistema com segurança. A organização em três pacotes bem delimitados (`lexer`, `parser` + `symbol`, `semantic`), a separação clara de responsabilidades entre a construção da Tabela de Símbolos (feita durante o parsing) e a sua utilização (feita na análise semântica), e a documentação explícita das limitações assumidas são, em conjunto, o que torna este projeto fácil de manter apesar da sua dimensão. Fica também demonstrado que é possível construir as três fases clássicas de um *front-end* de compilador — léxica, sintática e semântica — usando apenas as funcionalidades da biblioteca padrão de Java, sem qualquer dependência externa, o que facilita tanto a compreensão do código como a sua avaliação académica.

19. Referências Bibliográficas

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2ª edição. Boston: Pearson/Addison Wesley, 2006.

LOUDEN, Kenneth C. *Compiler Construction: Principles and Practice*. Boston: PWS Publishing Company, 1997.

APPEL, Andrew W. *Modern Compiler Implementation in Java*. 2ª edição. Cambridge: Cambridge University Press, 2002.

GOSLING, James; JOY, Bill; STEELE, Guy; BRACHA, Gilad; BUCKLEY, Alex; SMITH, Daniel; BIERMAN, Gavin. *The Java Language Specification*, edição correspondente ao Java SE 17. Oracle America, Inc.

PARR, Terence. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*.

Apêndice B — Manual do Utilizador

(ver documento completo a seguir, reproduzido integralmente nesta secção)

MANUAL DO UTILIZADOR

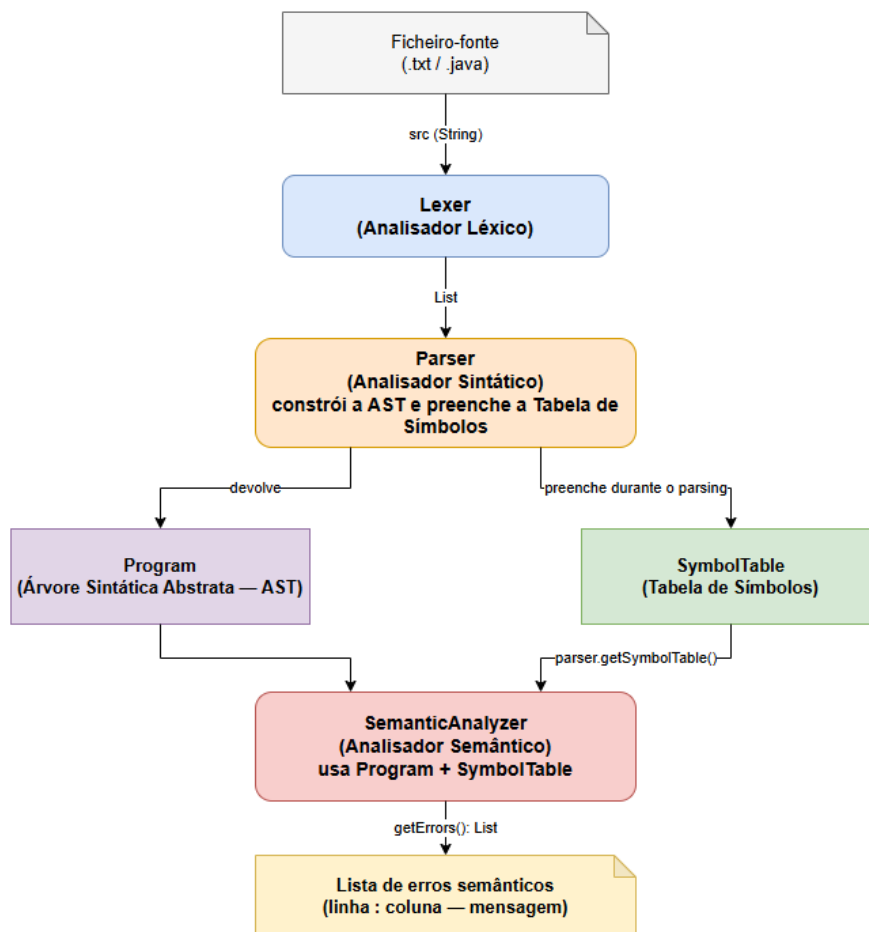
MINI-COMPILADOR para um subconjunto da linguagem Java

1. O QUE É ESTE PROGRAMA

Este MINI-COMPILADOR é uma ferramenta educativa que analisa ficheiros de texto escritos num **subconjunto da linguagem Java** e verifica se estão corretos em três níveis, um a seguir ao outro:

1. **Léxico** — se todos os "pedaços" do texto (palavras, números, símbolos) são reconhecidos.
2. **Sintático** — se esses pedaços estão organizados de acordo com a gramática da linguagem (chavetas fechadas, ponto-e-vírgula no sítio certo, etc.).
3. **Semântico** — se o programa "faz sentido": variáveis usadas foram declaradas, os tipos são compatíveis entre si, os métodos são chamados com o número e tipo de argumentos certos, etc.

O programa **não executa** o código Java nem gera um programa final — o seu único objetivo é **detetar e reportar erros**, indicando sempre a linha e a coluna onde cada problema ocorre, tal como um compilador real faz antes de gerar código.



2. REQUISITOS

- **Java Development Kit (JDK) versão 17 ou superior** instalado no computador.
- Um terminal (PowerShell, cmd, ou o terminal do NetBeans/VS Code).
- Um editor de texto simples para escrever os ficheiros de entrada (Bloco de Notas, Notepad++, VS Code, ou o próprio NetBeans).

Para confirmar que o Java está instalado, abra um terminal e execute:

```
java -version
```

Deve aparecer algo como `java version "17..."` ou superior.

3. INSTALAÇÃO

Não é necessária "instalação" no sentido tradicional — este é um projeto de código-fonte Java. Existem duas formas de o preparar:

Opção A — Abrir no NetBeans (mais simples)

1. Abrir o NetBeans.
2. File → Open Project... e selecionar a pasta `Compiler/`.
3. Clicar com o botão direito no projeto → **Build**.

Opção B — Compilar manualmente pela linha de comandos

Dentro da pasta `Compiler/`, execute:

```
Get-ChildItem -Recurse -Filter *.java src | ForEach-Object { $_.FullName } | Out-File  
sources.txt -Encoding utf8  
javac -d build\classes -encoding UTF-8 "@sources.txt"
```

Isto cria os ficheiros `.class` compilados dentro de `build\classes`.

4. COMO EXECUTAR

Depois de compilado, a partir da pasta onde ficaram as classes (por exemplo `build\classes`, ou `Compiler\build\classes` se usou o NetBeans):

```
java com.compiler.main.Main "caminho\para\o\seu\ficheiro.txt"
```

Por exemplo, para analisar um dos ficheiros de exemplo incluídos no projeto:

```
java com.compiler.main.Main "..\..\teste123.txt"
```

Se não indicar nenhum ficheiro, o programa tenta abrir por omissão um ficheiro chamado `teste_de_erro.txt` na pasta atual.

Sobre acentos na consola do Windows

Como as mensagens do programa estão em português (com "á", "ç", "õ", etc.), a consola do Windows pode, por predefinição, mostrar esses caracteres incorretamente

(ex.: **semântico** em vez de **semântico**). Para evitar isto, antes de correr o programa, execute uma vez por sessão de terminal:

```
chcp 65001
```

E execute o programa incluindo os parâmetros de codificação:

```
java "-Dfile.encoding=UTF-8" "-Dstdout.encoding=UTF-8" com.compiler.main.Main  
"ficheiro.txt"
```

5. FORMATO DO FICHEIRO DE ENTRADA

O ficheiro de entrada é um ficheiro de texto simples (extensão **.txt** ou **.java**, tanto faz — o programa só lê o conteúdo) que deve conter **uma classe ou interface Java** (ou várias, uma a seguir à outra), escrita seguindo as regras descritas na secção 7.

Exemplo mínimo válido:

```
public class Ola {  
    public static void main(String[] args) {  
        int x = 10;  
        System.out.println(x);  
    }  
}
```

6. COMO LER A SAÍDA DO PROGRAMA

Ao correr o programa, a saída aparece organizada em secções, sempre pela mesma ordem:

```
=== ERROS SEMANTICOS ===      (só aparece se houver erros)  
- lista de problemas semânticos encontrados, no formato:  
  "Erro semântico em <linha>:<coluna> - <descrição>"  
  
✓ COMPILAÇÃO CONCLUÍDA COM SUCESSO!      (a verde, se não houver  
erros)  
ou  
✗ COMPILAÇÃO CONCLUÍDA COM ERROS!      (a vermelho, com a contagem  
total)
```

Esta é a saída **focada no Analisador Semântico**, que é o objeto de avaliação deste exame: o programa mostra o resultado do parser, a Tabela de Símbolos completa (com todos os atributos exigidos pelo enunciado — endereço, tipo, dimensão, valor, escopo, tamanho em bytes) e a lista de erros semânticos, sem reimprimir o código-fonte ou o fluxo de tokens do Lexer, que não fazem parte do que este componente precisa de demonstrar.

6.1. Nota para quem quiser depurar o Lexer (avançado)

O código de `Main.java` mantém, comentados (não apagados), os blocos que reimprimiam o código-fonte lido e listavam cada token reconhecido pelo Lexer, um por linha, no formato `Token[TIPO | 'lexema' | linha:coluna]`. Se precisar de depurar especificamente a fase léxica, basta descomentar esses blocos em `Main.java` — a lista de tokens já é calculada internamente de qualquer forma, pois o `Parser` precisa dela; apenas a impressão está desativada por omissão.

6.2. A Tabela de Símbolos

Esta tabela lista **todas as declarações** encontradas no programa (classes, campos, parâmetros, variáveis locais, métodos, construtores), com estas colunas:

Coluna	Significado
Token	Categoria do símbolo (CLASS, FIELD, METHOD, PARAMETER, LOCAL_VARIABLE, ...)
Lexema	O nome do identificador
Linha	Linha onde foi declarado
Categoria	Descrição por extenso (ex.: "variável local")
Tipo de dado	<code>int</code> , <code>String</code> , <code>MinhaClasse</code> , etc.
Tipo de variável	Se é atributo, parâmetro ou variável local
Escopo	Onde a variável "vive" (ex.: <code>MinhaClasse.metodo</code>)
Valor	Valor inicial atribuído, se existir
Endereço	Endereço de memória simulado (fins didáticos, não é memória real)
Tamanho	Tamanho em bytes do tipo
Inicializado	<code>true/false</code>
Dim.	Número de dimensões, se for um array (<code>int[][] = 2</code>)
Parâmetros	Lista de parâmetros, se for um método/construtor
Modificadores	<code>public</code> , <code>static</code> , <code>final</code> , etc.
Retorno	Tipo de retorno, se for um método

6.3. Erros sintáticos vs. erros semânticos

- Um **erro sintático** significa que o código está "mal escrito" segundo a gramática (ex.: falta um `;`, uma chaveta não foi fechada). Estes erros são detetados primeiro.
- Um **erro semântico** significa que o código está bem escrito, mas não faz sentido logicamente (ex.: usar uma variável nunca declarada, comparar tipos incompatíveis, chamar um método com argumentos errados). Este programa continua a analisar semanticamente mesmo que já existam erros sintáticos noutra parte do ficheiro, para dar o máximo de informação possível numa só execução.

7. SUBCONJUNTO DA LINGUAGEM SUPORTADO

O compilador aceita um subconjunto amplo e realista de Java. Em resumo:

Estrutura do programa: `package`, `import` (incluindo `import static` e `import x.y.*`), declaração de classes e interfaces (com modificadores `public`, `abstract`, `final`, `static`, etc.), classes/interfaces aninhadas, parâmetros de tipo genérico na sintaxe (`<T extends X>`), `extends` e `implements`.

Membros de classe: campos (com múltiplos declaradores e inicializadores), métodos (com corpo ou abstratos), construtores, blocos de inicialização (estáticos ou de instância).

Tipos: todos os tipos primitivos (`boolean`, `byte`, `short`, `char`, `int`, `long`, `float`, `double`), `String`, `void`, tipos definidos pelo utilizador, arrays de qualquer dimensão (`int[]`, `String[][]`, ...).

Instruções: declaração de variáveis locais, `if/else` (incluindo encadeado), `while`, `do-while`, `for` (clássico e `for-each`), `switch` (com `case/default`), `break`, `continue`, `return`, `throw`, `try/catch/finally`, `assert`, `synchronized`, blocos `{ }`, instrução vazia `;`.

Expressões: todos os operadores aritméticos, relacionais, lógicos, bit a bit, de atribuição (simples e composta), `instanceof`, `?:` (ternário), incremento/decremento (pré e pós-fixos), `cast` (`Tipo`) `expr`, acesso a membros (`objeto.campo`, `objeto.metodo(...)`), acesso a arrays (`array[indice]`), criação de objetos (`new Classe(...)`), criação de arrays (`new int[10]`, com inicializador `{1,2,3}`), literais (inteiro, `long`, `float`, `double`, texto, caractere, `true/false/null`), `this`, `super`, comentários de linha (`//`) e de bloco (`/* */`, com aninhamento).

O que o analisador semântico verifica, especificamente

De acordo com o exigido para este exame, o programa verifica sempre:

1. **Variáveis não declaradas** — usar um nome que nunca foi declarado no escopo acessível.
2. **Declarações duplicadas** — declarar duas vezes o mesmo nome no mesmo escopo (ex.: dois campos `total` na mesma classe).

3. **Incompatibilidade de tipos** — usar operadores com tipos que não fazem sentido juntos (ex.: somar um `boolean` com um `int`).
4. **Tipos dos argumentos em chamadas de métodos** — número e tipo dos argumentos passados numa chamada devem coincidir com os parâmetros declarados.
5. **Atribuições incompatíveis** — atribuir um valor de um tipo a uma variável de outro tipo incompatível (ex.: atribuir uma `String` a um `int`).
6. **Condições das estruturas de controlo** — a condição de `if`, `while`, `do-while`, `for` e `assert` deve ser sempre do tipo `boolean`; a expressão de um `switch` deve ser `int/char/String`; o tipo do elemento de um `for-each` deve ser compatível com o array percorrido.

8. EXEMPLOS COMPLETOS PASSO A PASSO

8.1. Exemplo sem erros

Ficheiro `ola.txt`:

```
public class Ola {  
    public static void main(String[] args) {  
        int x = 10;  
        int y = 20;  
        if (x < y) {  
            System.out.println("x é menor que y");  
        }  
    }  
}
```

Ao correr `java com.compiler.main.Main ola.txt`, a saída termina com:

```
Parser executado com sucesso!  
...  
Analizador semântico executado sem erros!  
  
✓ COMPILAÇÃO CONCLUÍDA COM SUCESSO!  
✓ Nenhum erro encontrado durante a análise.
```

8.2. Exemplo com erros semânticos (todas as verificações)

Ficheiro `teste_semantico.txt` (incluído no projeto):

```
public class TesteSemantico {  
    private int total = 0;  
    private int total = 5;  
  
    public int soma(int a, int b) {  
        return a + b;  
    }  
}
```

```

public void executar() {
    int x = 10;
    String texto = "ola";

    int y = x + naoDeclarada;

    int z = texto;

    int resultado = soma(1, "dois");

    if (x) {
        System.out.println("nunca deveria compilar");
    }

    while (texto) {
        System.out.println("laço inválido");
    }

    for (int i = 0; texto; i++) {
        System.out.println(i);
    }
}
}

```

Ao correr o compilador sobre este ficheiro, a secção de erros semânticos mostra:

- Erro semântico em 3:25 - Símbolo 'total' já declarado no escopo 'TesteSemantico'
- Erro semântico em 13:21 - Variável 'naoDeclarada' não declarada
- Erro semântico em 15:17 - Não é possível atribuir um valor do tipo 'String' para a variável 'z' do tipo 'int'
- Erro semântico em 17:33 - Argumento 2 de 'soma' tem tipo 'String', esperado 'int'
- Erro semântico em 19:13 - Condição de 'if' deve ser do tipo boolean, mas é 'int'
- Erro semântico em 23:16 - Condição de 'while' deve ser do tipo boolean, mas é 'String'
- Erro semântico em 27:25 - Condição de 'for' deve ser do tipo boolean, mas é 'String'

X COMPILAÇÃO CONCLUÍDA COM ERROS!

X 7 erro(s) encontrado(s) durante a análise.

Repare como cada uma das seis exigências do enunciado está representada: duas declarações do campo `total` (b), uso de `naoDeclarada` (a), atribuição de `String` a `int` (c/e), argumento `"dois"` onde se esperava `int` (d), e três condições de controlo que não são `boolean` (f).

8.3. Exemplo com arrays e for-each

Ficheiro array_exemplo.txt:

```
public class ArrayExemplo {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3, 4, 5};

        for (int n : numeros) {
            System.out.println(n);
        }

        int soma = 0;
        for (int i = 0; i < numeros.length; i++) {
            soma = soma + numeros[i];
        }
        System.out.println(soma);
    }
}
```

Este exemplo está correto: o `for-each` percorre `numeros` (um `int[]`) com uma variável de laço `n` do tipo `int`, que é compatível com o tipo do elemento do array; o acesso `numeros[i]` usa um índice `int`, tal como exigido. A tabela de símbolos gerada mostra `numeros` com `Dim. = 1` (um array de uma dimensão) e o compilador termina sem erros.

Se, em vez disso, a variável do `for-each` fosse declarada com um tipo incompatível — por exemplo `for (String n : numeros)` — o compilador reportaria:

- Erro semântico em 5:22 - Elemento do tipo 'int' incompatível com a variável 'n' (String) do for-each

8.4. Exemplo com erro sintático

Ficheiro `teste_de_erro.txt` (incluído no projeto), propositadamente malformado (falta um `;`, um valor de atribuição e a classe nunca abre um bloco correto):

```
public class TesteErros
    private int x = ;
    private String nome
    private int y = 10
}
```

A saída mostra `Parser terminou com erro(s)`, mas recuperou e continuou a análise. seguido da lista de problemas sintáticos encontrados, cada um com linha e coluna — o programa não para na primeira ocorrência, tenta sincronizar e continuar a analisar o resto do ficheiro.

9. MENSAGENS DE ERRO MAIS COMUNS E O QUE SIGNIFICAM

Mensagem (exemplo)	O que significa	Como corrigir
Variável 'x' não declarada	Usou um nome que nunca foi declarado antes, nesse escopo	Declarar a variável antes de a usar, ou corrigir um erro de escrita no nome
Símbolo 'x' já declarado no escopo 'Y'	Duas declarações com o mesmo nome, no mesmo escopo	Remover ou renomear uma das duas declarações
Operador '+' não pode ser aplicado a tipos incompatíveis 'X' e 'Y'	Está a tentar usar um operador (+, -, &&, etc.) com tipos que não são compatíveis entre si	Verificar os tipos das duas variáveis/expressões envolvidas
Não é possível atribuir um valor do tipo 'X' para a variável 'v' do tipo 'Y'	O tipo do valor à direita do = não é compatível com o tipo declarado da variável	Corrigir o tipo da variável ou o valor atribuído
Argumento N de 'metodo' tem tipo 'X', esperado 'Y'	Um dos argumentos passados na chamada não tem o tipo esperado pelo parâmetro correspondente	Verificar a ordem e o tipo dos argumentos passados
Número de argumentos incorreto para 'metodo': esperado N, encontrado M	Chamou um método/construtor com um número de argumentos diferente do declarado	Verificar quantos parâmetros o método/construtor espera
Condição de 'if'/'while'/'for' deve ser do tipo boolean, mas é 'X'	A condição de uma estrutura de controlo não é uma expressão boolean	Trocar por uma comparação (x > 0, x == 5, etc.) em vez do valor "cru"
Método 'x' não existe na classe 'Y'	Está a chamar um método que não foi declarado nessa classe	Verificar o nome do método ou se a classe está definida no ficheiro
Campo 'x' não existe na classe 'Y'	Está a aceder a um campo que não foi declarado nessa classe	Verificar o nome do campo

10. PERGUNTAS FREQUENTES

P: Posso analisar vários ficheiros `.java` ao mesmo tempo? R: Não. O programa recebe um único caminho de ficheiro como argumento. Para analisar vários programas, execute o comando uma vez por ficheiro.

P: O programa executa o meu código Java? R: Não. Este é apenas o *front-end* de um compilador (análise léxica, sintática e semântica) — não há geração nem execução de código.

P: Porque é que dois construtores com o mesmo nome (sobrecarregados) dão erro de "já declarado"? R: Este mini-compilador, por desenho, não modela sobrecarga de métodos ou construtores — cada nome só pode ter uma assinatura por escopo (ver secção 11). Se precisar de comportamentos diferentes, use nomes de métodos diferentes.

P: Os meus `import`s de bibliotecas de terceiros dão erro? R: O programa não valida o conteúdo de imports — apenas os reconhece sintaticamente. Classes comuns do Java (`System`, `String`, `ArrayList`, `Math`, etc.) são reconhecidas por uma lista interna; classes verdadeiramente externas ao ficheiro-fonte não são validadas quanto aos seus membros.

P: Como sei se um erro é do analisador léxico, sintático ou semântico? R: Pela secção onde aparece na saída (ERROS SINTATICOS ou ERROS SEMANTICOS) — erros léxicos aparecem como avisos na consola ([AVISO] Caractere não reconhecido...) durante a fase de tokens.

11. LIMITAÇÕES E O QUE O PROGRAMA NÃO FAZ

- Não gera código executável nem um programa final — é apenas um verificador.
- Não modela sobrecarga de métodos/construtores (dois métodos com o mesmo nome no mesmo escopo são tratados como duplicados).
- Não modela herança entre classes definidas pelo próprio utilizador (apenas `Object` é tratado como super-tipo universal).
- Não valida os membros de classes externas à biblioteca padrão conhecida (apenas aceita o nome da classe).
- Não deteta erros de fluxo (ex.: "esta variável pode não ter sido inicializada neste caminho", "falta um `return` nalgum ramo").
- Os endereços de memória apresentados na tabela de símbolos são **simbólicos e sequenciais**, apenas para fins didáticos — não representam posições reais de memória.

12. RESOLUÇÃO DE PROBLEMAS (TROUBLESHOOTING)

Sintoma	Causa provável	Solução
Erro no parser: ... na consola e nada mais é impresso	Uma exceção inesperada interrompeu a análise (ex.: ficheiro vazio, ou combinação de sintaxe não coberta)	Confirmar que o ficheiro não está vazio e que é um Java válido de alto nível; testar com um dos ficheiros de exemplo primeiro
Erro <code>NoSuchFileException</code>	O caminho do ficheiro	Confirmar o caminho

ou semelhante	passado como argumento está incorreto	(relativo à pasta onde está a correr o java)
Caracteres acentuados aparecem como ? ou símbolos estranhos	Codificação da consola não está em UTF-8	Executar <code>chcp 65001</code> antes, e correr o java com <code>-Dfile.encoding=UTF-8 -Dstdout.encoding=UTF-8</code> (ver secção 4)
[AVISO] Caractere não reconhecido: 'X'	Existe no ficheiro-fonte um carácter que não pertence a nenhuma categoria de token (ex.: um emoji, ou um símbolo de outra linguagem)	Remover ou substituir o carácter no ficheiro-fonte
Uma classe/método que existe no ficheiro é reportado como "não declarada"	O identificador está a ser usado antes da sua classe estar registada corretamente, ou há um erro de escrita (maiúsculas/minúsculas importam em Java)	Confirmar a grafia exata (Java é <i>case-sensitive</i>) e que a classe está definida no mesmo ficheiro
O programa não compila (javac falha)	Versão do JDK inferior a 17	Instalar/usar o JDK 17 ou superior

13. GLOSSÁRIO DE TERMOS TÉCNICOS

Termo	Significado, em linguagem simples
Token	O menor "pedaço" reconhecível de um programa: uma palavra, um número, um símbolo. Ex.: <code>int</code> , <code>x</code> , <code>=</code> , <code>10</code> , <code>;</code> são cada um um token separado.
Lexema	O texto exato de um token, tal como aparece no ficheiro (ex.: o lexema do token "identificador" pode ser <code>contador</code>).
Analizador Léxico (Scanner/Lexer)	A parte do programa que lê o ficheiro carácter a carácter e o transforma numa lista de tokens.
Analizador Sintático (Parser)	A parte do programa que verifica se a sequência de tokens está organizada segundo as regras gramaticais da linguagem, construindo uma árvore que

	representa essa estrutura.
Árvore Sintática Abstrata (AST)	A representação em forma de árvore da estrutura do programa, produzida pelo analisador sintático (por exemplo, um if é representado como um nó com um ramo para a condição, um ramo para o "então" e um ramo opcional para o "senão").
Analisador Semântico	A parte do programa que verifica se o programa, apesar de gramaticalmente correto, também faz sentido (tipos compatíveis, variáveis declaradas, etc.).
Escopo	A "zona" do programa onde um nome (variável, parâmetro, etc.) está visível e pode ser usado. Por exemplo, uma variável declarada dentro de um método só é visível dentro desse método.
Tabela de Símbolos	Uma espécie de "ficha" com todas as declarações do programa (nome, tipo, escopo, endereço, etc.), usada pelo compilador para saber tudo sobre cada identificador.
Tipo (Type)	A categoria de um valor: int, boolean, String, uma classe do utilizador, um array, etc. Duas expressões só podem normalmente combinar-se (somar, comparar, atribuir) se os seus tipos forem compatíveis entre si.
Compatibilidade de tipos / atribuição (widening)	A regra que diz quando é seguro "converter automaticamente" um valor de um tipo para outro (por exemplo, um int pode ser atribuído a um double sem perda de informação, mas o inverso não é automático).
Erro léxico	Um problema detetado enquanto o programa ainda está a ser dividido em tokens (ex.: uma string que nunca fecha as aspas).
Erro sintático	Um problema de estrutura/gramática (ex.: falta um ; ou uma chaveta).
Erro semântico	Um problema de "significado" do

	programa (ex.: usar uma variável nunca declarada, ou somar um texto com um booleano).
Modo pânico (recuperação de erros)	A estratégia usada pelo compilador para, ao encontrar um erro, não parar imediatamente: avança no texto até um ponto seguro (normalmente um ; ou um }) e continua a analisar o resto do programa, reportando o máximo de erros possível numa só execução.

14. REFERÊNCIA RÁPIDA: PALAVRAS-RESERVADAS E OPERADORES RECONHECIDOS

14.1. Palavras-reservadas

```
abstract assert boolean break byte case catch
char class const continue default do double
else enum extends final finally float for
goto if implements import instanceof int
interface long native new package private
protected public return short static strictfp
super switch synchronized this throw throws
transient try void volatile while
```

(As palavras `const` e `goto` são reconhecidas como reservadas — tal como em Java — mas não têm nenhuma construção gramatical associada, exatamente como na própria linguagem Java.)

14.2. Literais especiais

`true`, `false`, `null`

14.3. Operadores e delimitadores

Categoria	Símbolos reconhecidos
Aritméticos	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>
Incremento/decremento	<code>++</code> <code>--</code>
Atribuição	<code>=</code> <code>+=</code> <code>-=</code> <code>*=</code> <code>/=</code> <code>%=</code> <code>&=</code> <code> =</code> <code>^=</code> <code><<=</code> <code>>>=</code> <code>>>>=</code>
Relacionais	<code><</code> <code>></code> <code><=</code> <code>>=</code> <code>==</code> <code>!=</code>
Lógicos	<code>&&</code> <code> </code> <code>!</code>

Bit a bit	& ^ ~
Deslocamento (shift)	<< >> >>>
Ternário	? :
Reticências (varargs)	...
Delimitadores	() { } [] ; , .

14.4. Categorias de literais

Categoria	Exemplos
Inteiro	0, 42, 1000
Long	42L, 1000l
Float	3.14f, 1.0F
Double	3.14159, 2.0d
Texto (String)	"Olá mundo!"
Caractere	'a', '\n'
Booleano / nulo	true, false, null

15. CONCLUSÃO

Este manual cobriu tudo o que é necessário para instalar, executar e interpretar corretamente os resultados do MINI-COMPILADOR: desde a preparação do ambiente (JDK 17+), passando pelo formato esperado do ficheiro de entrada e pela leitura de cada secção da saída (tokens, tabela de símbolos, erros sintáticos e semânticos), até aos exemplos práticos que ilustram cada uma das verificações semânticas implementadas. Com esta informação, qualquer utilizador — mesmo sem conhecimento prévio da implementação interna do compilador — consegue escrever um programa no subconjunto de Java suportado, correr o compilador sobre esse ficheiro, e compreender exatamente o que cada mensagem de erro reportada significa e como a corrigir.

16. REFERÊNCIAS BIBLIOGRÁFICAS

GOSLING, James; JOY, Bill; STEELE, Guy; BRACHA, Gilad; BUCKLEY, Alex; SMITH, Daniel; BIERMAN, Gavin. *The Java Language Specification*, edição correspondente ao Java SE 17. Oracle America, Inc.

AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compilers: Principles, Techniques, and Tools*. 2ª edição. Boston: Pearson/Addison Wesley, 2006.

Documentação oficial do JDK (Java Development Kit), Oracle. Disponível em: docs.oracle.com/en/java.